

Техническая спецификация: платформа SCORM-шаблонов

Статус: контракт формата шаблонов — источник истины для внутренних и внешних SCORM-шаблонов. Описывает действующее устройство: браузерный рендер `shell` и макетов, единый манифест, path-only DSL, `resultField` с рендерерами `core.*`, подгонку текста `textFit`, структурную валидацию и браузерную smoke-проверку, системные узлы раздела (`review`, `section-results`), варианты стартового экрана (`kind: "start"` со своим `layoutFile` и свойством страницы `image`) и страницу отчёта о результатах (`kind: "report" / report.adaptive`, §8.4) с постраничной раскладкой и принудительным разрывом листа (§8.4.6). За рамками формата — логика прохождения (задаётся сценарием теста, `flowPolicy`) и вычисляемые результаты (шкалы и показатели, публикуются в контекст рендера): см. §13 и §14. Настраиваемые надписи интерфейса итогов и порядок их подблоков — §8.3.1–8.3.3 и §10.5, разрез результата в карточке темы — §10.6, блоки разделов — §10.7, сводный разрез по тесту — §10.8. Ещё не действующие возможности вынесены в §3.

Версия: 1.7.0 (см. «История версий»)

Дата актуализации: 2026-08-15

История версий

Версия документа отражает состояние КОНТРАКТА формата шаблонов и версионруется семантически (`MAJOR.MINOR.PATCH`) — **отдельно** от `templateApiVersion` в манифесте (это версия рантайм-API шаблона, сейчас `1.0`, §5). Правила бампа:

- **MAJOR** — несовместимое изменение формата: ранее валидные шаблоны требуют правок (удалено/переименовано поле манифеста, изменён контракт слота, убран тип поля).
- **MINOR** — обратно-совместимое расширение: новый тип экрана, поля, параметра или настройки; прежние шаблоны продолжают работать без изменений.
- **PATCH** — уточнения формулировок, примеры, исправления опечаток без изменения контракта.

Версия	Дата	Изменения
1.7.0	2026-08-15	Три расширения одного трека (PRD-50, этапы Э2-Э5). Вердикт ключа: строка разреза несёт <code>passed</code> / <code>passClass</code> / <code>statusLabel</code> (§10.6) — <code>null</code> и пустые строки, когда порога у ключа нет или ключ не попал в выданное, поэтому строка никогда не объявляет провал, которого гейт не выносил. Блоки разделов: <code>result.topicGroups[]</code> со счётчиком «пройдено N из M» и <code>result.ungroupedTopics[]</code> (§10.7); плоский <code>topicResults</code> остаётся полным рядом с ними, поэтому шаблон печатает либо блоки, либо список. Сводный разрез по тесту: <code>result.breakdown[]</code> — одна строка на ключ по всей выдаче — приезжает пятым подблоком итогов (<code>breakdown</code> в <code>resultsBlockOrder</code> и <code>result.blocks[].isBreakdown</code> , §8.3.3 и §10.8), печатается и на адаптивных итогах, где карточка темы полос не несёт. Все три обратно-совместимы: шаблон, не печатающий новых полей, работает как прежде.
1.6.0	2026-08-14	Два расширения и одно уточнение. Разрез результата (PRD-50): строка темы несёт готовые подытоги по ключам оси — <code>result.topicResults[].breakdown[]</code> (§10.6): ключ, ширина полосы и подпись значения приходят посчитанными, показ включает автор теста настройкой содержания, а вердикт от неё не зависит. Принудительный разрыв страницы отчёта: пустой узел с атрибутом <code>data-page-break</code> в макете приказывает постраничной раскладке начать новый лист (§8.4.6). Уточнение к надписям: перечень надписей, которые редактор предлагает для ОТЧЁТА, считается сканированием макетов вариантов отчёта, а не объявляется в манифесте вторым списком (§8.3.2). Оба расширения обратно-совместимы: шаблон, не печатающий разрез и не ставящий метку разрыва, работает как прежде.
1.5.0	2026-08-12	Надписи интерфейса итогов объявляет ШАБЛОН, а не зашивает в макет (PRD-49): раздел манифеста <code>labels[]</code> (§8.3.1) с умолчаниями по экранам и хранением значения теста как ЗАПИСИ с тремя состояниями (§8.3.2); порядок и состав четырёх подблоков итогов — <code>resultsBlockOrder</code> по экранам (§8.3.3); контекст несёт разрешённое дерево <code>labels.*</code> и уже собранный массив подблоков <code>result.blocks[]</code> (§10.5). Расширение обратно-совместимое: шаблон, не объявивший <code>labels[]</code> , печатает собственные жёсткие строки макета, как и раньше.
1.4.0	2026-08-09	Контекст несёт ВВОДНЫЙ БЛОК <code>result.introHtml</code> (§10.3) — авторский текст, который печатается первым на экране итогов и в отчёте; тексты этих двух выдач задаются отдельно. Расширение обратно-совместимое: макет, не печатающий поле, работает как прежде.
1.3.0	2026-08-09	Поле варианта отчёта объявляет НАЗНАЧЕНИЕ — <code>settings[].scope : content</code> (что попадёт в документ; правится в «Настройках», рядом с обратной связью) или <code>appearance</code> (как документ выглядит; правится в «Оформлении») — см. §8.4.1. Расширение обратно-совместимое: умолчание <code>appearance</code> оставляет поля шаблона, не знающего о признаке, ровно там, где автор их находил.
1.2.0	2026-08-09	Форматирование авторского текста доведено до выдачи. В DSL добавлена интерполяция контролируемого HTML <code>{{& path }}</code> — единственный канал разметки внутри <code>{{#each}}</code> (§9); <code>{{{ path }}}</code> остаётся запрещённым. Контекст несёт разметку парными полями <code>textHtml</code> / <code>textsHtml</code> рядом с прежними строками (§10.3): ядро строит их из текста и его формата (<code>plain</code> экранируется, переводы строк становятся <code>
</code>; <code>richText</code> и <code>html</code> печатаются как есть). Расширение обратно-совместимое:

Версия	Дата	Изменения
		шаблон, печатающий только строку, работает как прежде, но показывает текст без форматирования.
1.1.0	2026-07-31	Отчёт о результатах вошёл в формат шаблона (PRD-27): виды <code>report</code> и <code>report.adaptive</code> в <code>contentTemplates[]</code> со своим <code>layoutFile</code> , необязательным <code>styleFile</code> и <code>settings[]</code> (§8.4); картинки отчёта (подложка, логотип) — файлы ШАБЛОНА, объявленные полями типа <code>image</code> (§8.4.1.1); контекст отчёта <code>report.*</code> (§10.2); среда стилей отчёта — контейнер вне сцены, корневой класс <code>tb-report</code> , скоупленный CSS (§8.4.3); статические проверки объявления и файлов варианта (§17.1, §17.2) и рендер отчёта в smoke-проверке (§17.3); хранение выбора варианта и значений полей — вне <code>design_settings_json</code> (§16). Расширение обратно-совместимое: шаблон, не объявивший вида отчёта, деградирует на макет стандартного шаблона.
1.0.0	2026-07-28	Первая версионированная редакция. Зафиксировано текущее состояние контракта: манифест и обязательные поля (§5), path-only DSL (§9), <code>resultField</code> с рендерерами <code>core.*</code> и <code>textFit</code> (§8.2.1.x), разделение <code>placeholders[]</code> / <code>settings[]</code> (§8.2.1), системные узлы раздела <code>review</code> / <code>section-results</code> (§8.2), две палитры и <code>themes[]</code> (§6), варианты стартового экрана (<code>kind: "start"</code> со своим <code>layoutFile</code>) и параметр иллюстрации <code>startImageUrl</code> (§8.2).

1. Назначение

Этот документ определяет общий технический контракт для внутренних и внешних SCORM-шаблонов. Он является источником истины для формата шаблонов: прочие документы опираются на него, а не переопределяют механику шаблонов независимо.

Платформа поддерживает:

- единую механику для встроенных и загружаемых шаблонов;
- исполнение шаблона только в браузере внутри сгенерированного SCORM-пакета;
- полную настройку макетов страниц, а не только цветов и шрифтов;
- контролируемую ответственность Core за оценку, состояние навигации, SCORM-состояние и стандартные интерактивы вопросов;
- расширение через `template.js` и Runtime API.

2. Базовые принципы

2.1 Исполнение только в браузере

Шаблоны исполняются только в браузере обучающегося внутри SCORM-пакета.

Сервер не выполняет HTML или JavaScript шаблона во время экспорта. Сервер только:

- валидирует ZIP загруженного шаблона;

- хранит метаданные шаблона;
- копирует выбранный шаблон в сгенерированный SCORM ZIP;
- добавляет в пакет данные теста и ресурсы Core runtime.

2.2 Единая механика внутренних и внешних шаблонов

Встроенные и внешние шаблоны используют одинаковые:

- структуру `manifest.json`;
- структуру ZIP/файлов;
- браузерный рендерер;
- контракт макетов;
- Runtime API;
- процесс валидации и smoke-проверки.

Физическое хранение может отличаться, но runtime-поведение одинаково.

2.3 Ответственность Core и шаблона

Core владеет:

- состоянием теста;
- состоянием навигации;
- состоянием ответов;
- оценкой;
- состоянием обратной связи;
- восстановлением сессии;
- интеграцией с SCORM API;
- telemetry;
- рендерерами стандартных интерактивов вопросов;
- финальными защитами для критичных действий.

Шаблон владеет:

- макетом оболочки;
- макетами страниц;
- визуальным представлением;
- опциональным клиентским поведением через `template.js`.

3. Перспективные изменения

Раздел собирает возможности, которые ещё не входят в действующий формат: пока шаблон на них полагаться не может. При внедрении такая возможность переходит в основные разделы, а формат получает новую версию (см. «История версий»).

3.1 Расширенные интерактивы вопросов

Переопределение механики стандартных интерактивов средствами `template.js`.

Предполагаемая регистрация рендерера:

```
TestBuilder.template.registerInteractionRenderer("single", renderer);
TestBuilder.template.registerInteractionRenderer("multiple", renderer);
TestBuilder.template.registerInteractionRenderer("matching", renderer);
TestBuilder.template.registerInteractionRenderer("ranking", renderer);
```

Контракт рендерера:

```
{
  render(ctx),
  mount(root, ctx),
  getAnswer(root),
  setAnswer(root, answer),
  clearAnswer(root),
  setLocked(root, locked),
  showCorrectState(root, ctx),
  destroy(root)
}
```

Форматы ответов совпадают с теми, что принимает Core:

```
single: 2
multiple: [0, 3]
matching: { "0": 2, "1": 0 }
ranking: [2, 0, 1, 3]
```

3.2 Закрепление файловых версий шаблона

Выбранная версия шаблона (`templateVersion`) сохраняется для диагностики и миграций; хранение каждой прежней файловой версии не требуется. Возможное развитие — закрепление теста за точной файловой версией шаблона.

3.3 Система `renderer`-плагинов

Расширяемый реестр рендереров показателей результата помимо встроенных `core.*` (описан в §8.2.1.3 как перспективный).

4. Структура ZIP шаблона

Рекомендуемая структура:

```
template-id/  
  manifest.json  
  shell.html  
  preview.png  
  demo/  
    course.json  
  
  layouts/  
    start.html  
    question.html  
    content.html  
    results.html  
    question-single.html  
    system-locked.html  
    report.html  
    report.adaptive.html  
  
  partials/  
    topic-nav.html  
    progress.html  
    header.html  
    footer-actions.html  
  
  styles/  
    base.css  
    theme.css  
    report.css  
  
  scripts/  
    template.js  
  
  assets/  
    fonts/  
    images/  
    icons/
```

`manifest.json` является источником истины. Дополнительные файлы разрешены, но все файлы, на которые есть ссылки, должны существовать внутри ZIP.

Внешние URL запрещены. Все CSS, JS, шрифты, изображения, иконки и другие ресурсы должны быть локальными файлами внутри SCORM ZIP.

Сгенерированный SCORM-пакет включает выбранный шаблон, а не полную библиотеку шаблонов. Кроме него в пакет попадают резервные макеты и стили стандартного шаблона — но только для системных экранов, которые выбранный шаблон не объявляет.

5. Манифест

5.1 Базовая структура

```
{
  "id": "corporate",
  "name": "Corporate",
  "version": "1.0.0",
  "templateApiVersion": "1.0",
  "description": "",

  "layouts": {
    "shell": "shell.html",
    "question": "layouts/question.html",
    "content": "layouts/content.html",
    "results": "layouts/results.html",
    "start": "layouts/start.html",
    "review": "layouts/review.html",
    "section-results": "layouts/section-results.html",
    "system.blocked": "layouts/system-blocked.html",
    "report": "layouts/report.html",
    "report.adaptive": "layouts/report.adaptive.html"
  },

  "contentTemplates": [
    {
      "key": "intro.hero",
      "label": "Введение: крупный заголовок и изображение",
      "kind": "intro",
      "pageKind": "content.intro",
      "layoutFile": "layouts/content-intro.html",
      "placeholders": [
        {
          "key": "title",
          "type": "text",
          "label": "Заголовок",
          "required": true,
          "maxLength": 120,
          "textFit": {
            "mode": "autoFitFont",
            "defaultFontSize": 36,
            "minFontSize": 24,
```



```
        "maxFontSize": 36,
        "allowAuthorFontSize": false,
        "overflow": "warn"
    }
},
{
    "key": "subtitle",
    "type": "text",
    "label": "Подзаголовок",
    "required": false,
    "maxLength": 500,
    "textFit": {
        "mode": "fixed",
        "defaultFontSize": 20,
        "allowAuthorFontSize": true,
        "allowedFontSizes": [16, 18, 20, 22],
        "overflow": "warn"
    }
},
{
    "key": "heroImage",
    "type": "image",
    "label": "Изображение",
    "required": false
}
]
},
{
    "key": "info.textWithImage",
    "label": "Информация: текст и изображение",
    "kind": "info",
    "pageKind": "content.info",
    "layoutFile": "layouts/content-info.html",
    "placeholders": [
        { "key": "title", "type": "text", "label": "Заголовок", "required": true },
        { "key": "lead", "type": "text", "label": "Вводный текст", "required": false },
        { "key": "body", "type": "richText", "label": "Основной текст", "required": true },
        { "key": "image", "type": "image", "label": "Изображение", "required": false }
    ]
},
{
    "key": "summary.progressRing",
    "label": "Итог: кольцевая диаграмма прогресса",
```

```

"kind": "summary",
"pageKind": "content.summary",
"layoutFile": "layouts/content-summary.html",
"placeholders": [
  { "key": "title", "type": "text", "label": "Заголовок", "required": true },
  {
    "key": "progressChart",
    "type": "resultField",
    "label": "Показатель прогресса",
    "required": true,
    "allowedPaths": [
      "progress.active.percent",
      "progress.question.percent",
      "progress.page.percent",
      "result.scorePercent",
      "sectionResult.percent"
    ],
    "defaultPath": "progress.active.percent",
    "allowedRenderers": ["core.ringChart", "core.progressBar", "core.textMetric"],
    "defaultRenderer": "core.ringChart"
  }
],
},
{
  "key": "report.standard",
  "label": "Отчёт: стандартный",
  "kind": "report",
  "layoutFile": "layouts/report.html",
  "styleFile": "styles/report.css",
  "isDefault": true,
  "settings": [
    { "key": "headline", "type": "text", "label": "Заголовок отчёта", "default": "Итоги" },
    { "key": "backgroundImage", "type": "image", "label": "Подложка страницы" }
  ]
},
{
  "key": "report.adaptive.standard",
  "label": "Отчёт: уровни",
  "kind": "report.adaptive",
  "layoutFile": "layouts/report.adaptive.html",
  "styleFile": "styles/report.css",
  "isDefault": true,

```

```
    "settings": []
  }
],

"rendererPlugins": [
  {
    "key": "core",
    "version": "1.0.0",
    "source": "core",
    "renderers": ["textMetric", "badge", "progressBar", "ringChart", "segmentedProgress"]
  }
],

"systemPages": [
  {
    "id": "system.locked",
    "layout": "layouts/system-locked.html",
    "purpose": "blocking",
    "countInProgress": false,
    "allowNavBack": false
  }
],

"partials": {
  "topicNav": "partials/topic-nav.html",
  "progress": "partials/progress.html",
  "header": "partials/header.html",
  "footerActions": "partials/footer-actions.html"
},

"assets": {
  "styles": ["styles/base.css", "styles/theme.css"],
  "scripts": ["scripts/template.js"],
  "fonts": ["assets/fonts/Inter.woff2"],
  "images": ["assets/images/logo.svg"],
  "preview": "preview.png"
},

"preview": {
  "demoData": "demo/course.json",
  "defaultRoute": "start",
  "routes": [
    "start",
```

```

        "content.intro",
        "content.info",
        "question.single",
        "question.multiple",
        "question.matching",
        "question.ranking",
        "content.summary",
        "results",
        "system.locked"
    ],
    "viewports": ["desktop", "mobile"]
},

"params": [
    {
        "key": "brand.primaryColor",
        "type": "color",
        "label": "Основной цвет",
        "default": "#0066cc",
        "group": "Бренд",
        "cssVar": "--tb-brand-primary",
        "validation": {
            "required": true
        }
    }
],

"capabilities": {
    "navigation": ["linear", "free", "locked"],
    "sidebar": true,
    "progress": ["questions", "pages"],
    "timer": true,
    "contentPages": ["start", "intro", "info", "summary", "html"],
    "questionTypes": ["single", "multiple", "matching", "ranking"],
    "customInteractions": false,
    "runtimeApi": "1.0"
}
}

```

5.2 Обязательные поля манифеста

Обязательные:

- `id`
- `name`
- `version`
- `templateApiVersion`
- `contentTemplates` (непустой массив, минимум один элемент — требует схема манифеста)
- `layouts.shell`
- `layouts.question`
- `layouts.content`
- `layouts.results`
- `assets`
- `assets.preview`
- `preview`
- `params`
- `capabilities`

Опциональные:

- `description`
- `layouts.start`
- детализированные макеты вопросов, например `question.single`
- детализированные макеты контентных страниц, например `content.info`
- `rendererPlugins`
- `partials`
- `systemPages`
- варианты отчёта (`contentTemplates[]` с `kind: "report" / report.adaptive` , §8.4) и парные им ключи `layouts.report / layouts["report.adaptive"]`
- `labels[]` — надписи интерфейса итогов, доступные автору для переформулировки/выключения (§8.3.1)
- `resultsBlockOrder` — состав и порядок подблоков итогов по экранам (§8.3.3)

Отчёт объявлять не обязательно: шаблон без вариантов отчёта его не теряет — страница собирается макетом стандартного шаблона с оформлением активного (§8.4.4). Надписи и порядок подблоков объявлять тоже не обязательно: шаблон без `labels[]` печатает свои жёсткие строки макета, а без `resultsBlockOrder` получает зашитый порядок «Общий балл, По шкалам, По показателям, По темам» (§8.3.3).

5.3 Резервный выбор макетов

Core выбирает макет сначала по самому специфичному ключу, затем откатывается к общему ключу.

Примеры:

```

question.single    → question
question.multiple  → question
content.intro      → content
content.info       → content
content.summary    → content
content.html       → content
start             → content
system.blocked     → system page layout → content
report             → layoutFile варианта → макет вида из стандартного шаблона
report.adaptive    → layoutFile варианта → макет вида из стандартного шаблона

```

У отчёта отката на общий макет HET: осмысленного «макета отчёта по умолчанию» внутри шаблона не существует, поэтому вариант обязан объявить свой `layoutFile`, а единственный резерв — одноимённый вид стандартного шаблона (§8.4.4).

Обязательный минимум:

```

{
  "layouts": {
    "shell": "shell.html",
    "question": "layouts/question.html",
    "content": "layouts/content.html",
    "results": "layouts/results.html"
  }
}

```

5.4 Системные страницы

Шаблоны могут объявлять системные страницы для неучебных экранов: блокировки, предупреждения, промежуточные экраны, сертификаты или экраны после результата.

Поддерживаемые значения `purpose`:

```

blocking
warning
interstitial
postResult
certificate
custom

```

Системные страницы не влияют на оценку. `countInProgress` и `allowNavBack` определяют поведение прогресса и навигации.

Реализация. Встроенный `default` объявляет системные экраны не массивом `systemPages[]`, а ключами в `layouts: system.blocked, system.transition` (адаптивный переход), а также узлы раздела `review, section-results, section-intro`. Полный массив `systemPages[]` с `purpose / countInProgress / allowNavBack` схемно не валидируется, хотя входит в контракт.

5.5 Контракт предпросмотра и демонстрационного набора данных

Предпросмотр является частью контракта шаблона, а не произвольной страницей шаблона. Он нужен для галереи, проверки загружаемых ZIP и ручной оценки поведения шаблона до активации.

Шаблон обязан объявить:

- статический предпросмотр в `assets.preview`;
- живой предпросмотр в `preview`;
- демонстрационный набор данных в `preview.demoData`.

`assets.preview` - локальный путь внутри ZIP к `png, jpg, jpeg, webp` или `svg`. Файл используется в галерее и списке шаблонов. Внешние URL запрещены.

`preview` описывает Core-owned live preview. Шаблон не должен полагаться на отдельный `preview.html` как на контрактный endpoint: такой файл может существовать как вспомогательный стенд разработчика, но админский UI и smoke-проверка запускают шаблон через `shell.html`, манифест, `layouts`, ресурсы и демонстрационный набор.

```

type TemplatePreviewContract = {
  demoData: string;
  defaultRoute?: PreviewRoute;
  routes?: PreviewTarget[];
  viewports?: Array<"desktop" | "tablet" | "mobile">;
};

type PreviewTarget =
  | PreviewRoute
  | {
    route: PreviewRoute;
    label?: string;
    pageId?: string;
    templateKey?: string;
  };

type PreviewRoute =
  | "start"
  | "content.intro"
  | "content.info"
  | "content.summary"
  | "content.html"
  | "question.single"
  | "question.multiple"
  | "question.matching"
  | "question.ranking"
  | "review"
  | "section-results"
  | "results"
  | `system.${string}`;

```

Требования к `preview`:

- `demoData` указывает на локальный JSON-файл внутри ZIP;
- `defaultRoute` определяет первый экран живого предпросмотра, по умолчанию `start`;
- `routes` перечисляет экраны, которые UI и smoke-проверка должны открыть; если поле не задано, Core строит список из обязательных возможностей манифеста;
- если в пакете несколько `contentTemplates[]` одного `pageKind`, `routes[]` должен различать их через `templateKey` или `pageId`;
- `viewports` задаёт контрольные размеры предпросмотра; если поле не задано, Core проверяет `desktop` и `mobile`;

- все маршруты должны соответствовать объявленным `layouts`, `contentTemplates`, `systemPages` и `capabilities`;
- live preview работает без SCORM API, LMS, сетевых запросов и внешних ресурсов.

Демонстрационный набор данных - это стабильный JSON, совместимый с публичным runtime context. Он должен покрывать все маршруты из `preview.routes`, все заявленные типы вопросов, динамические `resultField` placeholders, progress, результаты и системные страницы.

Минимальный формат:

```
type PreviewDemoDataset = {
  schemaVersion: "1.0";
  locale?: string;
  params?: Record<string, unknown>;
  course: {
    title: string;
    mode?: "standard" | "adaptive";
    navigation?: "linear" | "free" | "locked";
    topics: PreviewTopic[];
    contentPages: PreviewContentPage[];
    questions: PreviewQuestion[];
  };
  runtime: {
    route: PreviewRoute;
    progress: {
      active: PreviewProgress;
      question: PreviewProgress;
      page: PreviewProgress;
    };
    result: PreviewResult;
    sectionResult?: PreviewResult;
    state?: Record<string, unknown>;
  };
};

type PreviewTopic = {
  id: string;
  title: string;
  status?: "locked" | "available" | "completed";
};

type PreviewContentPage = {
  id: string;
  type: "start" | "intro" | "info" | "summary" | "html";
  route?: PreviewRoute;
  templateKey?: string;
  topicId?: string;
  values: Record<string, unknown>;
};

type PreviewQuestion = {
  id: string;
```

```

topicId?: string;
type: "single" | "multiple" | "matching" | "ranking";
prompt: string;
options?: Array<{ id: string; text: string; correct?: boolean }>;
pairs?: Array<{ id: string; left: string; right: string }>;
order?: string[];
feedback?: {
  text?: string;
  correctAnswerPublic?: Record<string, unknown>;
};
};

type PreviewProgress = {
  current: number;
  total: number;
  percent: number;
};

type PreviewResult = {
  score: number;
  maxScore: number;
  scorePercent: number;
  status: "notStarted" | "inProgress" | "passed" | "failed" | "partial";
};

```

Требования к значениям:

- `schemaVersion` обязателен; несовместимая версия блокирует активацию шаблона;
- `params` переопределяет значения по умолчанию из `manifest.params` только для preview;
- для каждого `manifest.contentTemplates[]` должна быть хотя бы одна демонстрационная `contentPages[]` с тем же `templateKey`;
- `contentPages[].values` должен соответствовать `placeholders[]` выбранного `templateKey`;
- значения `resultField` должны использовать только `allowedPaths`, `allowedRenderers` и валидные `rendererOptions`;
- `questions[]` должен содержать по одному примеру для каждого типа из `capabilities.questionTypes`;
- `progress.*.percent`, `result.scorePercent` и `sectionResult.scorePercent` должны находиться в диапазоне `0..100`;
- демонстрационные данные не должны содержать персональные данные реальных пользователей, production ID, токены, LMS-поля или сетевые URL;
- Core может использовать один и тот же demo dataset для авторского preview и smoke-проверки.

При запуске live preview Core:

1. загружает `manifest.json`;
2. валидирует `assets.preview`, `preview` и `preview.demoData`;
3. строит публичный runtime context из `PreviewDemoDataset`;
4. рендерит `defaultRoute` через обычный runtime pipeline;
5. даёт UI возможность переключать маршруты из `preview.routes`;
6. логирует diagnostics с route, viewport, renderer key и ошибкой, если rendering/fallback был задействован.

Ошибки контракта preview делятся так:

- отсутствующий `assets.preview`, `preview` или `preview.demoData` - блокирующая ошибка;
- невалидный JSON demo dataset - блокирующая ошибка;
- route, для которого нет layout/template/capability, - блокирующая ошибка;
- отсутствие покрытия необязательного route - предупреждение;
- fallback renderer в preview - предупреждение, если страница осталась работоспособной;
- необработанная ошибка runtime в preview - блокирующая ошибка.

6. Параметры шаблона

Шаблоны определяют параметры через расширенный `params[]` с dot-keys, группами, значениями по умолчанию и валидацией.

Поддерживаемые типы:

```
color
text
number
boolean
select
image
font
asset
```

Пример:

```
{
  "key": "progress.mode",
  "type": "select",
  "label": "Прогресс",
  "options": ["questions", "pages", "hidden"],
  "default": "questions",
  "group": "Навигация"
}
```

Семантика `progress.mode`

Параметр `progress.mode` управляет тем, что считается единицей прогресса в runtime.

Режим	Единица прогресса	Знаменатель
<code>questions</code>	Каждый вопрос, на который дан ответ	Общее число вопросов в тесте
<code>pages</code>	Каждый переход навигации (<code>Core.next()</code>)	Число <code>navigable</code> -экранов, фиксируется при загрузке
<code>hidden</code>	Не отображается	—

Правила подсчёта для `pages` :

- Знаменатель фиксируется при инициализации flow и не меняется в процессе прохождения.
- Каждый вызов `Core.next()` , приводящий к переходу на новый экран, увеличивает числитель на 1.
- Страницы `start` и `results` включены в подсчёт — они являются частью навигационного flow.
- Каждый вопрос считается отдельной единицей, а не весь блок вопросов темы: это обеспечивает плавный рост прогресса без скачков.
- Страница `system.blocked` исключена: её показ не изменяет счётчик прогресса. Blocked-состояние не является частью нормального flow.
- В режиме `pages` `progress.active.*` совпадает с `progress.page.*` ; в режиме `questions` — с `progress.question.*` .

Core передаёт параметры в контекст макета, а также генерирует CSS-переменные (CSS custom properties) в браузере для параметров с `cssVar` .

Пример:

```
{
  "key": "brand.primaryColor",
  "type": "color",
  "default": "#0066cc",
  "cssVar": "--tb-brand-primary"
}
```

CSS, сгенерированный в браузере:

```
<style id="tb-template-vars">
  :root {
    --tb-brand-primary: #0066cc;
  }
</style>
```

6.1 Темы шаблона

Шаблон может поставлять несколько палитр. Стилями это выражается блоками `theme.css` (`:root`, `@media (prefers-color-scheme: dark)`, `:root[data-theme="dark"]`), но платформе этого НЕ достаточно: палитры объявляются в манифесте.

```
"themes": [
  { "id": "light", "label": "Светлая" },
  { "id": "dark", "label": "Тёмная" }
]
```

Поле	Обязательность	Смысл
<code>themes</code>	нет	Палитры шаблона. Нет поля или меньше двух записей — платформа считает, что тем нет
<code>themes[].id</code>	да	Идентификатор из закрытого перечня платформы: <code>light</code> , <code>dark</code>
<code>themes[].label</code>	да	Подпись пункта переключателя и колонки в таблице цветов

Перечень закрыт так же, как реестр типов полей (§8.2.1): идентификатор вне перечня — ошибка шаблона, а не молча пропущенная запись. Проверка выполняется при загрузке И при активации; при загрузке дополнительно сверяется, что у каждой объявленной темы есть хотя бы один свой токен в стилях.

Что делает Core:

- **Выбор вида теста.** Автор фиксирует палитру (`light / dark`) или отдаёт её системной настройке участника (`auto`). Значение хранится у теста, а не у шаблона.
- **Значение цвета на каждую палитру.** Цветовые параметры хранятся отдельно по темам; нецветовые — одним значением, они от палитры не зависят.
- **Постановка темы на сцену.** Закреплённая палитра ставится атрибутом `data-theme` на корень сцены: `<html>` в SCORM-пакете, `shadow-host` на веб-хосте. При `auto` атрибут НЕ ставится — иначе медиа-запрос шаблона не получит своей очереди.
- **Печать переопределений.** Правила той же структуры, что и в `theme.css` , добавляются ПОСЛЕ стилей шаблона: специфичность равна, побеждает порядок.

На веб-хосте условие обязано идти функциональной формой — `:host([data-theme="dark"])` , `:host(:not([data-theme="light"]))` . Суффиксная форма `:host[data-theme="dark"]` невалидна, и браузер отбрасывает правило целиком: тема молча перестаёт применяться. По той же причине при переносе стилей шаблона в теневой корень условие переносится ВНУТРЬ скобок.

6.2 Миграция параметров при обновлении шаблона

Правило:

- тот же `key` и тот же `type` -> сохранить значение;
- старый ключ отсутствует -> игнорировать/удалить;
- новый ключ с `default` -> использовать `default` ;
- новый ключ без `default` -> автор должен заполнить значение.

UI должен показывать отчёт:

2 параметра сохранены, 1 параметр больше не используется, 1 новый параметр требует значения.

Будущее расширение: явные `paramMigrations` в манифесте для сложных enterprise-шаблонов.

7. Контракт оболочки

`shell.html` определяет внешний макет плеера.

Обязательные элементы оболочки:

```
<div data-slot="page"></div>

<button data-nav="next"></button>
<button data-action="answer-submit"></button>
<button data-action="test-finish"></button>
```

Реализация. Структурный валидатор и браузерная проверка работоспособности требуют от оболочки только `data-slot="page"` (код ошибки `SHELL_CONTRACT`). Маркеры `data-nav` / `data-action` привязываются Core, когда присутствуют, но **не являются** обязательными для прохождения валидации/проверки: встроенный `default` их в `shell.html` не объявляет, а действия Core навешивает делегированием по `[data-action]` поверх экранов, отрисованных `renderScreenInto`. Жёсткая проверка привязки `next/ answer-submit/test-finish` относится к перспективным расширенным интерактивам (§3).

Опциональные элементы оболочки:

```
<button data-nav="prev"></button>
<div data-slot="progress"></div>
<div data-slot="topic-nav"></div>
<div data-slot="feedback"></div>
<div data-slot="timer"></div>
<div data-slot="breadcrumb"></div>
```

Оболочка может предоставлять резервные кнопки действий. Макеты страниц также могут объявлять локальные кнопки. Если для страницы существуют локальные действия, Core может скрыть или отключить резервные кнопки оболочки на этой странице.

7.1 Монтирование оболочки (`mountShell`)

По умолчанию Core монтирует экраны в собственный базовый контейнер, а `shell.html` служит только внешним каркасом. Флаг манифеста `"mountShell": true` включает ПРЯМОЕ монтирование оболочки: Core подставляет `shell.html` целиком и рендерит каждый экран внутри её `#app` (`data-slot="page"`), заменяя базовый контейнер.

Флаг обязателен для шаблонов с ФИКС-СЦЕНОЙ — когда CSS шаблона привязан к структуре оболочки (например `.tb-frame > .tb-stage > #app`, сцена фиксированного размера или на `container-query` единицах `cqh / cqw`). Без `mountShell` такой шаблон рендерится вне своей сцены, и его оформление не применяется. То же монтирование выполняет и веб-хост в предпросмотре: оба хоста читают `mountShell` из манифеста, поэтому автор видит фикс-сцену одинаково в SCORM и в превью. Шаблон без фикс-сцены (как встроенный `default`) флаг не объявляет.

Пример локального действия:

```
<button data-nav="next" data-local-action="true">
  <span>{{ nav.nextLabel }}</span>
</button>
```


Core привязывает обработчики и обновляет состояние. Он не должен перезаписывать пользовательский HTML кнопки, если кнопка не пустая.

Core управляет состоянием кнопок навигации/действий через:

- `hidden`
- `disabled`
- `aria-disabled`
- `data-state`
- `data-visible`
- классы состояния, например `is-hidden` и `is-disabled`

8. Контракт макетов страниц

8.1 Макеты вопросов

Обязательные слоты:

```
<div data-slot="question-text"></div>
<div data-slot="question-interaction"></div>
```

Реализация. Текст вопроса монтируется в `data-slot="question-text"` — это и есть имя слота, которое проверяет валидатор (код `QUESTION_CONTRACT`) и заполняет рендерер (`question-text` + `question-interaction`, см. `server/scorm/templates/default/layouts/question.html`). Имя `question-prompt` из ранних черновиков спецификации устарело; источник истины — `question-text`.

Опциональные слоты:

```
<div data-slot="question-media"></div>
<div data-slot="question-feedback"></div>
<div data-slot="question-hint"></div>
<div data-slot="question-counter"></div>
<div data-slot="question-topic"></div>
<div data-slot="question-difficulty"></div>
```

Core вставляет:

- формулировку вопроса в `question-text`;
- стандартный рендерер интерактива в `question-interaction`;

- медиа в `question-media` и обратную связь в `question-feedback`, если эти слоты присутствуют.

Реализация. Из опциональных слотов рантайм реально заполняет только `question-media` и `question-feedback`. Слоты `question-hint` / `question-counter` / `question-topic` / `question-difficulty` управляемыми НЕ являются: подсказка приходит внутри `question-interaction` (элемент с классом `question-hint`), а счётчик выводится через `data-path` (`state.questionCounterLabel`).

8.2 Макеты и шаблоны контентных страниц

Контентные страницы работают по модели PowerPoint slide layouts:

- **тип страницы** определяет смысл страницы в flow: `intro`, `info`, `summary`, `html`, `blocked`;
- **content template** в SCORM-шаблоне определяет скелет данных страницы: placeholders, их типы, обязательность, ограничения и layout;
- **экземпляр страницы в тесте** хранит значения placeholders, которые заполнил автор;
- **layout** отображает эти значения, но не является источником содержимого.

Это означает, что автор не заливает произвольную вёрстку в шаблонную страницу. Автор выбирает подходящий скелет страницы и заполняет его структурированные поля. Свободный HTML остаётся отдельным типом `content.html` для технических сценариев и не является основным способом создания страниц.

Поддерживаемые режимы определения страницы:

Mode	Назначение	Когда использовать
<code>template</code>	Страница создаётся по <code>manifest.contentTemplates[]</code> ; автор заполняет placeholders	Основной режим для стандартизированных корпоративных страниц
<code>standard</code>	Страница использует каноническую схему Core для <code>intro/info/summary</code> и fallback layout	Для переносимых страниц, не завязанных на конкретный шаблон
<code>html</code>	Страница содержит санитизированный HTML и отображается через <code>content.html</code>	Только как escape hatch для технических авторов

Поддерживаемые kinds контентных страниц:

Откуда экран берёт макет — зависит от его класса (см. классы ниже таблицы). Системные экраны резолвятся по фиксированным ключам `layouts[]`; прочие контентные страницы — по `layoutFile` варианта, иначе по общему `layouts["content"]`. Когда шаблон не объявляет нужный системный экран, применяется fallback на стандартный шаблон с предупреждением в UI — колонка «Fallback» ниже.

Отдельного вида страницы «галерея» больше нет: галерея — обычный вариант информационной страницы (`kind: "info"`) со своим `layoutFile` , и отдельного правила резолвинга для неё не существует. Индикатор навигации по подряд идущим страницам макет рисует из вычисленного контекста `page.*` (§10), а не из значений, введённых автором. Свойства страницы (подпись кнопки `nextLabel` , фон `backgroundImage` , идентификатор последовательности `sequenceId`) объявляются отдельным блоком `settings[]` варианта — по природе они не содержимое. Когда вариант, к которому привязана страница, в выбранном шаблоне отсутствует, страница рендерится вариантом по умолчанию своего `kind` (см. §8.2.2), а `templateKey` в базе не переписывается: замену подтверждает автор через диалог сопоставления. Оба поставляемых шаблона несут сетку из шести раскладок контентной страницы в двух семействах (обычная и галерейная) — двенадцать вариантов; их перечень — эталонный, но не нормативный.

Резолвинг макета делится на три класса (по фактическому рантайму `server/scorm/template/app/render/`).

- **Системные экраны** (`results` , `question` , `review` , `section-results` , «Введение раздела», `system.blocked`) резолвятся по ФИКСИРОВАННОМУ ключу `layouts[<key>]` . Вариант в `contentTemplates[]` для них не задаёт макет — он нужен лишь для привязки страницы и для определения fallback (объявлен ли `kind`). В частности «Введение раздела» рендерится по ключу `layouts["section-intro"]` , а НЕ по `layoutFile` варианта `intro` ; если этого ключа нет, экран падает в общий контентный рендер.
- **Стартовый экран** (`start`) резолвится по `layoutFile` ВЫБРАННОГО варианта старта, иначе по фиксированному `layouts["start"]` . Шаблон может объявить несколько `contentTemplates[]` с `kind: "start"` , каждый со своим `layoutFile` (напр. `start.image-right` — колонка-иллюстрация справа); базовый помечается `isDefault: true` . Автор выбирает раскладку через «Сменить вариант» на строке «Старт», и ОБА хоста рендерят выбранный макет (SCORM-рантайм `startPage.resolveStartLayout` , веб-хост `GET /screen-template/start`). Отсутствие выбранного варианта или его файла — fallback на `layouts["start"]` (и далее на стандартный шаблон). Контекст строит общий `buildStartState` независимо от раскладки; иллюстрацию несёт СВОЙСТВО СТРАНИЦЫ `image` того варианта, который его объявил (`settings[]`), — вариант без свойства иллюстрации не получает. Значение попадает в контекст как `design.startImageUrl` через общий `startImageForVariant` .
- **Прочие контентные страницы** (`info` / `summary` / `router` / `html`) резолвятся по `layoutFile` варианта, иначе по общему `layouts["content"]` . Галерея — частный случай `info` и отдельного правила не имеет.

Page kind	Назначение	Откуда макет	Fallback на стандартный шаблон
<code>start</code>	Стартовый экран теста (лендинг) — тест-уровневый, всегда	<code>layoutFile</code> выбранного варианта старта, иначе <code>layouts["start"]</code>	Да
<code>question</code>	Экран вопроса	<code>layouts["question"]</code>	Да
<code>results</code>	«Итоги теста» — итоговый результат всего теста, тест-уровневый, всегда	<code>layouts["results"]</code>	Да
<code>review</code>	«Обзор раздела/теста» перед завершением — системный узел	<code>layouts["review"]</code>	Да
<code>section-results</code>	Вычисляемые «Итоги раздела» — системный узел	<code>layouts["section-results"]</code>	Да
<code>content.intro</code>	«Введение раздела» (<code>before_topic</code>) — по одной на тему	<code>layouts["section-intro"]</code> , иначе общий контентный рендер	Да
<code>system.blocked</code>	Системная страница блокировки	<code>systemPages[].layout</code> / <code>layouts["system.blocked"]</code>	Да (файловый)
<code>content.info</code>	Информационная/ учебная страница; сюда же относится слайд галереи — вариант со своим <code>layoutFile</code>	<code>layoutFile</code> варианта, иначе <code>layouts["content"]</code>	Нет: недоступный вариант подменяется вариантом по умолчанию <code>kind</code> , замену подтверждает автор (§8.2.2)
<code>content.summary</code>	«Итог раздела» (<code>after_topic</code>) — по одной на тему; показывает результат РАЗДЕЛА	<code>layoutFile</code> варианта, иначе <code>layouts["content"]</code>	Нет: вариант по умолчанию <code>kind</code> (§8.2.2)
<code>content.router</code>	Меню тем (<code>router_by_topics</code>)	<code>layoutFile</code> варианта, иначе <code>layouts["content"]</code>	Нет: вариант по умолчанию <code>kind</code> (§8.2.2)
<code>content.html</code>	Санитизированный HTML-блок	<code>layouts["content"]</code>	Нет: общий контентный рендер

Fallback на стандартный шаблон применяется только к экранам с ВЫДЕЛЕННЫМ макетом (`start` , `results` , `question` , `intro` , `review` , `section-results` , `system.blocked`). Экраны, которые рендерятся общим `layouts["content"]` (`info` / `summary` / `router` / `html`), подменять нечем — их «резервом» служит сам общий контентный макет.

Отчёт о результатах (`report` , `report.adaptive`) в таблице выше не значится намеренно: это не экран прохождения, а документ, который обучающийся скачивает файлом. Объявляется он тем же `contentTemplates[]` , но живёт по своим правилам — свой резолвинг макета, своя среда стилей и свои проверки. Полный контракт — в §8.4.

`start` и `results` — тест-уровневые системные экраны (по одному на тест, в любом режиме). `review` и `section-results` — системные узлы раздела: «Обзор» перед завершением и вычисляемые «Итоги раздела» после него; они рендерятся своими рантайм-фазами и исключены из потока контентных страниц. `content.intro` — «Введение раздела» перед его вопросами (по одной на тему, только в режимах по темам). `content.summary` — устаревшая per-topic закладка «Итог раздела»: остаётся валидной для обратной совместимости, но её роль выполняет вычисляемый `section-results` .

8.2.1 `contentTemplates[]`

`manifest.contentTemplates[]` объявляет скелеты страниц, доступные автору при выбранном шаблоне. Один SCORM-шаблонный пакет может содержать несколько content templates. Это штатный механизм для вариантов страниц: например несколько `content.info` , несколько `content.summary` , разные варианты стартового экрана (`start`) или итогов теста (`results`). Стандартный (`default`) шаблон обязан объявить хотя бы по одному варианту каждого системного `kind`: `start` , `results` , `router` , `questions` , `intro` , `review` , `section-results` , а также по одному варианту каждого вида отчёта — `report` и `report.adaptive` (он — системный fallback; `review` и `section-results` — узлы раздела; `summary` остаётся валидным `kind` для обратной совместимости, но в обязательный набор уже не входит).

Множественность работает так:

- `contentTemplates[]` - массив, а не одиночный объект;
- `contentTemplates[].key` уникален внутри `manifest.contentTemplates[]` ;
- каждый `contentTemplates[]` элемент обязан иметь человекочитаемое `label` ;
- несколько templates могут иметь одинаковый `pageKind` , если у них разные `key` ;
- несколько templates могут использовать один и тот же layout, если их placeholders совместимы с этим layout;
- авторский UI при добавлении страницы показывает все доступные templates текущего пакета и может группировать их по `pageKind` ;
- авторский UI показывает автору `label` , а не технический `key` ;
- `templateKey` экземпляра страницы всегда указывает на конкретный `contentTemplates[].key` , а не только на `pageKind` или layout.

Минимальная структура:

```
{
  "key": "info.textWithImage",
  "label": "Информация: текст и изображение",
  "kind": "info",
  "pageKind": "content.info",
  "layoutFile": "layouts/content-info.html",
  "description": "Страница с заголовком, основным текстом и иллюстрацией",
  "placeholders": [
    {
      "key": "title",
      "type": "text",
      "label": "Заголовок",
      "required": true,
      "maxLength": 120,
      "textFit": {
        "mode": "fixed",
        "defaultFontSize": 32,
        "allowAuthorFontSize": false,
        "overflow": "error"
      }
    },
    {
      "key": "body",
      "type": "richText",
      "label": "Основной текст",
      "required": true,
      "textFit": {
        "mode": "autoFitFont",
        "defaultFontSize": 20,
        "minFontSize": 14,
        "maxFontSize": 20,
        "allowAuthorFontSize": false
      }
    },
    {
      "key": "image",
      "type": "image",
      "label": "Изображение",
      "required": false,
      "constraints": {
        "aspectRatio": "16:9"
      }
    }
  ]
}
```

```
}  
]  
}
```

Реализация. Схема манифеста (`contentTemplateEntrySchema`) требует у элемента `key` , `label` и `kind` (виды из §8.2); `pageKind` и `placeholders[]` опциональны. Путь к макету указывается полем `layoutFile` — относительным путём к файлу, напр. `"layouts/content-info.html"` . Ключа `layout` в контракте нет. `layoutFile` читается рантаймом только для НЕсистемных контентных страниц (`info` / `summary` / `router` / `html`); системные экраны и «Введение раздела» резолвятся по фиксированным ключам `layouts[]` (см. таблицу выше). См. эталонный `server/scorm/templates/default/manifest.json` .

Типы `placeholders` — ЗАКРЫТЫЙ перечень. Placeholder описывает СОДЕРЖИМОЕ, вставляемое в макет:

```
text  
textarea  
richText  
html  
image  
resultField
```

Перечень приведён к фактически поддерживаемому и объявлен однократно в общем коде (`shared/template/field-types.ts`): он питает и контрол редактирования, и правило отрисовки значения, а привязка «тип — контрол» задаётся полным отображением, поэтому пропуск контрола становится ошибкой сборки, а не молчаливой деградацией поля в однострочный ввод. Из прежнего списка `number` , `boolean` и `select` переезжают в `settings[]` (см. ниже) — по природе это настройки страницы, а не её содержимое; `video` , `file` и `actionLabel` не поддерживаются и из контракта исключены. Неизвестный тип — ошибка валидации (§17.1).

`settings[]` — свойства страницы

Кроме содержимого вариант может объявить СВОЙСТВА страницы — то, что управляет её поведением и оформлением, но не вставляется в макет как контент. Раздел необязателен: его отсутствие означает, что у страниц этого варианта настраиваемых свойств нет.

```

{
  "key": "gallery.card",
  "label": "Галерея: карточка",
  "kind": "info",
  "layoutFile": "layouts/gallery.html",
  "placeholders": [
    { "key": "header", "type": "text", "label": "Заголовок" },
    { "key": "cardText", "type": "richText", "label": "Текст карточки" }
  ],
  "settings": [
    { "key": "sequenceId", "type": "sequence", "label": "Идентификатор последовательности"
  },
    { "key": "nextLabel", "type": "text", "label": "Подпись кнопки", "default": "Далее" },
    { "key": "backgroundImage", "type": "image", "label": "Фон страницы" }
  ]
}

```

Типы настроек — закрытый перечень: `number`, `boolean`, `select`, `text`, `image`, `sequence`.

Правила:

- настройка может объявлять значение по умолчанию (`default`) и обязательность (`required`); значение по умолчанию подставляется при создании страницы, а незаполненная обязательная настройка блокирует сохранение так же, как незаполненный обязательный placeholder;
- редактор структуры не имеет собственных полей: форма страницы состоит ИСКЛЮЧИТЕЛЬНО из объявленных `placeholders[]` и `settings[]`;
- `sequence` — идентификатор последовательности страниц. Подряд идущие страницы одного участка структуры с одинаковым непустым значением образуют последовательность; ядро само вычисляет число точек индикатора и текущую позицию и передаёт их макету через `page.*` (§10). Вариант, не объявивший `sequence`, поля не показывает и точек не получает;
- значения настроек хранятся отдельно от содержимого страницы и вместе с ним попадают в SCORM-пакет и в снимок публикации;
- добавление раздела обратно совместимо: `templateApiVersion` не повышается.

Вариант может пометить себя `isDefault: true` — это делает его вариантом по умолчанию для своего `kind` (см. §8.2.2). Если пометки нет ни на одном варианте `kind`, по умолчанию считается ПЕРВЫЙ объявленный вариант этого `kind`.

8.2.2 Недоступный вариант: подмена вариантом по умолчанию

Автор привязывает страницу к варианту (`templateKey`). Сменив шаблон оформления теста, автор может получить страницы, чей вариант новый шаблон не объявляет. Такая страница не ломается и не исчезает: она рендерится ВАРИАНТОМ ПО УМОЛЧАНИЮ своего `kind` (см. выше). Правило

едино для всех трёх мест, где страница превращается в экран, — SCORM-рантайм, веб-хост и предпросмотр — и живёт в общем коде (`shared/template/content-page.ts` → `resolveContentTemplate`).

Подмена — решение ОТРИСОВКИ, а не правки данных:

- `templateKey` в базе не переписывается. Поэтому привязка переживает возврат прежнего шаблона, а «Структура» продолжает пометить страницу как требующую сопоставления;
- окончательную замену подтверждает автор через диалог «Сменить вариант», который показывает теряемые при переходе значения. До подтверждения — только подмена при отрисовке;
- если шаблон не объявляет НИ ОДНОГО варианта нужного `kind` , подменять нечем — страница деградирует до простого контентного макета (заголовок и текст), как и раньше;
- список тестов пометает жёлтым знаком `test`, в котором есть хотя бы одна страница с недоступным вариантом (число — в подсказке, по клику — переход в «Структуру»). Признак считается на сервере за один запрос на всю видимую страницу списка, а не по запросу на тест.

8.2.1.1 Политика размера текста

Для placeholders типов `text` , `textarea` и `richText` шаблон должен явно определить поведение текста при переполнении блока. Модель следует логике PowerPoint text box:

Реализация. Подгонку размера текста (`autoFitFont` , `growBox` , `overflow`) реально применяет только SCORM-рантайм (`applyTextFit` в `templateCore.js` , покрыто тестами). Веб-хост (единый рендерер `renderScreenInto`) `textFit` пока НЕ применяет — там это только метаданные плейсхолдера. Значение `allowAuthorFontSize` /ручной размер сохраняются на сервере независимо от хоста.

<code>textFit.mode</code>	Поведение	Когда использовать
<code>fixed</code>	Размер шрифта и размер блока фиксированы; переполнение диагностируется/обрезается согласно <code>overflow</code>	Для строгих брендовых слайдов
<code>autoFitFont</code>	Core уменьшает размер шрифта в пределах <code>minFontSize</code> / <code>maxFontSize</code> , чтобы текст поместился	Для заголовков и коротких блоков
<code>growBox</code>	Размер шрифта фиксирован, а блок увеличивается по вертикали под текст в рамках <code>layout constraints</code>	Для длинных информационных блоков

Пример:

```

{
  "key": "title",
  "type": "text",
  "label": "Заголовок",
  "required": true,
  "maxLength": 120,
  "textFit": {
    "mode": "autoFitFont",
    "defaultFontSize": 36,
    "minFontSize": 24,
    "maxFontSize": 36,
    "allowAuthorFontSize": false,
    "overflow": "warn"
  }
}

```

Поля `textFit`:

Поле	Назначение
<code>mode</code>	<code>fixed</code> , <code>autoFitFont</code> , <code>growBox</code>
<code>defaultFontSize</code>	Размер шрифта по умолчанию из шаблона
<code>minFontSize</code>	Минимальный размер для <code>autoFitFont</code>
<code>maxFontSize</code>	Максимальный размер для <code>autoFitFont</code> и ручного ввода
<code>allowAuthorFontSize</code>	Разрешает автору вручную менять размер шрифта для этого placeholder
<code>allowedFontSizes</code>	Допустимые значения ручного размера, если нужен список вместо диапазона
<code>overflow</code>	<code>warn</code> , <code>clip</code> , <code>scroll</code> , <code>error</code>
<code>maxHeight</code>	Максимальная высота блока для <code>growBox</code> , если layout допускает ограничение

Требования к `textFit`:

- если `allowAuthorFontSize = false`, UI не показывает управление размером шрифта для этого placeholder;
- если `allowAuthorFontSize = true`, UI ограничивает размер шрифта диапазоном `minFontSize / maxFontSize` или отдельным `allowedFontSizes`;
- ручной размер шрифта сохраняется как `override` конкретной страницы, а не как изменение layout шаблона;
- `autoFitFont` выполняется Core/runtime, а не произвольным кодом шаблона;
- при `fixed` и переполнении Core должен применить `overflow` и записать диагностику;

- при `growBox` Core может увеличивать вертикальный размер блока только в пределах ограничений `layout`, чтобы не ломать соседние элементы.

Ручной размер шрифта хранится отдельно от текстового значения:

```
{
  "templateKey": "intro.hero",
  "values": {
    "title": "Перед началом раздела",
    "subtitle": "Короткое описание"
  },
  "placeholderStyles": {
    "subtitle": {
      "fontSize": 18
    }
  }
}
```

`placeholderStyles[placeholderKey].fontSize` валиден только если соответствующий `placeholder` имеет `allowAuthorFontSize = true`. Для `richText` ручной размер применяется к контейнеру `placeholder`; произвольные `inline font-size` внутри `richText` не поддерживаются.

Общие требования к `contentTemplates[]`:

- `contentTemplates[].key` уникален внутри `manifest.contentTemplates[]`;
- `contentTemplates[].label` обязателен, не пустой, человекочитаемый и пригоден для отображения в UI выбора страницы, `preview` и диагностике;
- `contentTemplates[].label` не должен быть техническим идентификатором, `path` или повторением `key`; если нужно пояснение, используется опциональный `description`;
- `placeholders[].key` уникален внутри конкретного `contentTemplate`;
- `placeholders[].label` обязателен для каждого поля, которое показывается автору в форме;
- `type`, `required`, `maxLength`, `options`, `constraints` используются UI для формы заполнения;
- для `text` / `textarea` / `richText` задана `textFit` -политика; если не задана, Core использует `fixed` и пишет предупреждение в валидации шаблона;
- `richText` хранится как структурированный документ или ограниченный HTML, прошедший санитизацию;
- `image/video/file` ссылаются на локальные `assets`, которые упаковываются в SCORM ZIP;
- `layout` не должен ожидать значения, не объявленные в `placeholders`.

Объём допустимого HTML в значениях автора

Разметку принимают только типы `richText` и `html`; значения прочих типов экранируются при отрисовке. Санитизация СЕРВЕРНАЯ и выполняется дважды: при сохранении страницы (автор получает список удалённого) и повторно при сборке SCORM-пакета; рендерер вставляет очищенное значение без дополнительной обработки.

Модель — чёрный список. Удаляются: `<script>`, `<iframe>`, `<svg>`, `<object>`, `<embed>`, `<link>`, `<meta>` (вместе с содержимым), все атрибуты-обработчики `on*`, `javascript:` в `href / src` (значение заменяется на `#`) и `src / href` на внешний `http(s)://` (атрибут удаляется целиком — пакет обязан быть автономным, поэтому ссылка на внешний сайт в тексте не сохраняется). Остальная разметка проходит: форматирование, структура, таблицы, контейнеры, атрибуты `class / id / style`, внутренние ссылки и изображения из `/uploads/`.

Ограничения, которые обязан учитывать шаблон: тег вне списка запрещённых пройдёт (включая `<style>`, `<form>`, `<input>`), `data: -URI` не блокируются, разбор ведётся регулярными выражениями без DOM. Поэтому макет и его CSS рассчитаны на чужую разметку: оформление изолируется классами компонентов и не опирается на глобальные селекторы.

Объём допустимого HTML не меняется — он равен описанному выше; дополнительные перечни разрешённых или запрещённых тегов не вводятся. Меняется ввод: у текстового поля появляется переключатель режимов (простой текст / форматированный текст / HTML), где ТИП поля задаёт потолок (`html` — все три режима, `richText` — первые два, `text / textarea` — только простой). В режиме форматированного текста работает единая для всех шаблонов панель (полужирный, курсив, зачёркнутый, списки, ссылка, очистка), а вставка из внешнего источника нормализуется; в режиме HTML запрещённая конструкция НЕ сохраняется — выдаётся ошибка со списком найденного. Объявления `allowedMarks / allowedBlocks` из контракта исключены. Ресурсы шаблона (изображения, стили) адресуются относительным путём (`images/logo.png`); базовый путь подставляет ядро на каждом хосте, внешние `http(s)` -ссылки остаются запрещёнными.

8.2.1.2 Динамические placeholders и визуализация показателей

Динамический контент страницы отображается через placeholder типа `resultField`. Шаблон объявляет, какие runtime-пути и какие контролируемые renderers допустимы, а автор выбирает источник данных и подпись. Шаблон не вычисляет результат и не получает произвольный JavaScript для диаграмм.

Пример декларации:

```
{
  "key": "progressChart",
  "type": "resultField",
  "label": "Показатель прогресса",
  "required": true,
  "allowedPaths": [
    "progress.active.percent",
    "progress.question.percent",
    "progress.page.percent",
    "result.scorePercent",
    "sectionResult.percent"
  ],
  "defaultPath": "progress.active.percent",
  "allowedRenderers": ["ringChart", "progressBar", "segmentedProgress", "textMetric"],
  "defaultRenderer": "ringChart",
  "format": {
    "type": "percent",
    "decimals": 0
  }
}
```

Экземпляр страницы хранит выбор автора:

```
{
  "templateKey": "summary.progressRing",
  "values": {
    "title": "Ваш прогресс",
    "progressChart": {
      "path": "progress.active.percent",
      "renderer": "ringChart",
      "label": "Пройдено"
    }
  }
}
```

Layout размещает только placeholder:

```
<h1>{{ page.values.title }}</h1>
<div data-placeholder="progressChart"></div>
```

Core выполняет безопасный pipeline:

1. валидирует `path` по `allowedPaths` ;
2. читает значение из публичного runtime context;
3. нормализует значение по `format` ;
4. валидирует `renderer` по `allowedRenderers` и registry renderer plugins;
5. вызывает renderer через контролируемый runtime API;
6. вставляет результат в `data-placeholder` .

Стандартные runtime-пути:

Path	Назначение
<code>progress.active.current</code> / <code>progress.active.total</code> / <code>progress.active.percent</code>	Активный прогресс согласно <code>progress.mode</code>
<code>progress.question.current</code> / <code>progress.question.total</code> / <code>progress.question.percent</code>	Прогресс только по вопросам
<code>progress.page.current</code> / <code>progress.page.total</code> / <code>progress.page.percent</code>	Прогресс по всем страницам flow
<code>result.scoreRaw</code> / <code>result.scoreMax</code> / <code>result.scorePercent</code>	Итоговый результат теста
<code>result.status</code>	Итоговый статус, например <code>passed</code> / <code>failed</code>
<code>sectionResult.scoreRaw</code> / <code>sectionResult.scoreMax</code> / <code>sectionResult.percent</code>	Результат текущего раздела для <code>content.summary</code>
<code>sectionResult.status</code>	Статус текущего раздела
<code>result.{name}</code>	Пользовательские показатели результата
<code>retake.availableDate</code>	Дата доступного повторного прохождения для <code>system.blocked</code>
<code>retake.effectiveToday</code>	Нормализованное «сегодня», от которого считаются производные величины; равно <code>todayDate</code> , если часы не пришлось нормализовать

Стандартные renderers поставляются как plugin `core` и используются по полным ключам `core.textMetric` , `core.ringChart` и т.д.

Renderer	Назначение	Рекомендация UI/UX
<code>core.textMetric</code>	Крупное число/значение с подписью	Для точного результата, статуса, даты
<code>core.badge</code>	Компактный статус	Для <code>passed / failed</code> , уровня, зоны риска
<code>core.progressBar</code>	Линейная полоса прогресса	Лучший default для процесса прохождения
<code>core.ringChart</code>	Кольцевая диаграмма одного процента	Для одного hero-показателя, не использовать пачками
<code>core.segmentedProgress</code>	Полоса из сегментов по количеству шагов/вопросов	Для небольшого числа вопросов или разделов
<code>core.questionTiles</code>	Строка/сетка кубиков: каждый кубик = вопрос	Для диагностического прогресса, когда $total \leq 40$; при большем числе нужна агрегация
<code>core.sectionList</code>	Список разделов со статусами/процентами	Для тестов с явными разделами
<code>core.scaleBars</code>	Несколько горизонтальных шкал	Для компетенций и многошкальных результатов

Реализация. Реально зарегистрированы пять рендереров: `core.textMetric`, `core.badge`, `core.progressBar`, `core.ringChart`, `core.segmentedProgress` (`shared/template/renderers.ts`). `core.questionTiles`, `core.sectionList`, `core.scaleBars` в действующий набор не входят (перспективные, §3). Реестр устойчив к ошибкам: рендерер не из `allowedRenderers`, неизвестный или упавший откатывается к `core.textMetric`; `path` не из `allowedPaths` даёт пустое значение.

Рекомендации:

- для обычного текущего прогресса использовать `progressBar`;
- для стартовой/итоговой страницы с одним главным числом использовать `ringChart` + текстовый процент;
- для тестов до 40 вопросов можно использовать `questionTiles`; для больших тестов использовать `segmentedProgress` с группировкой или `progressBar`;
- цвет не должен быть единственным носителем смысла: `renderer` обязан выводить текст/label или доступный `aria-label`;
- `thresholds` цветов задаются шаблоном/`rendererOptions`, а не произвольной логикой автора;
- динамические `renderers` не влияют на результат, SCORM-статусы и навигацию.

8.2.1.3 Renderer plugins

Renderer plugin - расширение, которое добавляет один или несколько контролируемых renderers для `resultField`. Он отвечает только за визуальное представление уже рассчитанных данных. Plugin не может менять ответы, результат, SCORM-статусы, навигацию, `TEST_DATA` или runtime state.

Полноценная система плагинов ниже относится к перспективным изменениям (§3) и пока не действует: доступны только встроенные `core.*` (см. §8.2.1.2). SCORM-рантайм содержит рудиментарный загрузчик `manifest.rendererPlugins` (инъект `<script>` по `plugin.path / src`) и хук регистрации `TestBuilder.renderers.register(id, fn)`, где рендерер — чистая функция `(value, options) ⇒ string`, а не `mount()`-инстанс из контракта ниже. Реестр/ `source` (`core / template / registry`), проверка `version`, валидация `optionsSchema` и тип `RendererRegistry / DynamicRenderer` НЕ реализованы. Веб-хост (единственный рендерер) загрузки плагинов не имеет вовсе — там доступны только пять `core.*`. Эталон объявляет `"rendererPlugins": []`.

Источники renderer plugins:

Source	Назначение
<code>core</code>	Встроенные renderers платформы, доступны всегда
<code>template</code>	Renderer plugin, поставляемый внутри конкретного SCORM-шаблона
<code>registry</code>	Администрируемый plugin из общего registry, копируется в SCORM ZIP при экспорте

Манифест шаблона или экспортированного пакета объявляет используемые plugins:


```

{
  "rendererPlugins": [
    {
      "key": "core",
      "version": "1.0.0",
      "source": "core",
      "renderers": ["textMetric", "badge", "progressBar", "ringChart"]
    },
    {
      "key": "rtk.progress",
      "version": "1.2.0",
      "source": "template",
      "entry": "renderers/progress/index.js",
      "styles": ["renderers/progress/style.css"],
      "renderers": [
        {
          "key": "questionTiles",
          "label": "Кубики вопросов",
          "valueTypes": ["progress"],
          "optionsSchema": {
            "type": "object",
            "properties": {
              "maxTiles": { "type": "number", "default": 40 },
              "shape": { "type": "string", "enum": ["square", "rounded"] }
            }
          }
        }
      ]
    }
  ]
}

```

Полный ключ renderer строится как `{pluginKey}.{rendererKey}` . Например:

```

{
  "allowedRenderers": ["core.progressBar", "core.ringChart", "rtk.progress.questionTiles"],
  "defaultRenderer": "core.progressBar"
}

```

Runtime API plugin:

```

type DynamicRendererPlugin = {
    key: string;
    version: string;
    register(registry: RendererRegistry): void;
};

type RendererRegistry = {
    register(renderer: DynamicRenderer): void;
};

type DynamicRenderer = {
    key: string;
    label: string;
    valueTypes: Array<"number" | "percent" | "string" | "boolean" | "date" | "progress" |
"list">;
    optionsSchema?: JsonSchema;
    mount(root: HTMLElement, input: RendererInput): RendererInstance | void;
};

type RendererInput = {
    value: unknown;
    rawValue: unknown;
    path: string;
    label?: string;
    format?: Record<string, unknown>;
    options?: Record<string, unknown>;
    context: PublicRuntimeContext;
    theme: RuntimeTheme;
};

type RendererInstance = {
    update?(input: RendererInput): void;
    destroy?(): void;
};

```

Ограничения безопасности и совместимости:

- renderer plugin загружается только из SCORM ZIP, внешние URL запрещены без отдельной политики;
- `eval`, `Function`, inline script injection и запись в глобальные объекты Core запрещены;
- plugin не имеет доступа к SCORM API напрямую;
- plugin получает только публичный runtime context и не должен читать приватные данные;

- plugin обязан обрабатывать пустое/ `null` значение и показывать fallback;
- ошибка plugin не должна ломать страницу: Core заменяет renderer на fallback `core.textMetric` или показывает диагностику placeholder;
- `options` автора валидируются по `optionsSchema` до сохранения и перед runtime;
- renderer обязан поддерживать доступность: текстовая подпись, `aria-label`, отсутствие зависимости только от цвета;
- если plugin не найден или версия несовместима, страница остаётся открываемой с fallback renderer и диагностикой.

UI автора для `resultField` должен:

1. показать только renderers, перечисленные в `allowedRenderers` и доступные в registry;
2. после выбора renderer построить форму `rendererOptions` по `optionsSchema`;
3. показывать preview на демонстрационных данных;
4. сохранять выбор в значении placeholder:

```
{
  "path": "progress.question.percent",
  "renderer": "rtk.progress.questionTiles",
  "label": "Вопросы",
  "rendererOptions": {
    "maxTiles": 30,
    "shape": "square"
  }
}
```

8.2.2 Как автор определяет страницу

Основной сценарий:

1. Автор нажимает **“Добавить страницу”**.
2. Выбирает место в flow.
3. Выбирает режим:
 - **по шаблону** (`template`);
 - **стандартная страница** (`standard`);
 - **HTML-страница** (`html`).
4. Для режима `template` выбирает один из `manifest.contentTemplates[]` текущего SCORM-шаблона.
5. UI строит форму по `placeholders[]`.
6. Автор заполняет значения placeholders.
7. Предпросмотр показывает страницу через layout выбранного шаблона.
8. При сохранении в тест записываются `templateKey`, `pageKind`, `values`, позиция и служебные настройки.

Пример экземпляра страницы:

```
{
  "id": "page-1",
  "mode": "template",
  "templateKey": "info.textWithImage",
  "pageKind": "content.info",
  "topicId": "topic-1",
  "position": "before_topic",
  "values": {
    "title": "Перед началом раздела",
    "body": {
      "format": "richText",
      "document": {}
    },
    "image": {
      "assetId": "asset-123",
      "src": "assets/content/page-1.png"
    }
  },
  "autoAdvance": false
}
```

Runtime page получает значения в `page.values`:

```
{
  "id": "page-1",
  "kind": "content.info",
  "layoutKey": "content.info",
  "templateKey": "info.textWithImage",
  "values": {
    "title": "Перед началом раздела",
    "body": {},
    "image": {
      "src": "assets/content/page-1.png"
    }
  }
}
```

Layout использует значения как обычные path-only переменные:

```
<h1>{{ page.values.title }}</h1>
<div data-placeholder="body"></div>
<img data-placeholder="image">
```

Core заполняет `data-placeholder` согласно типу placeholder. Для `richText` Core вставляет только санитизированный результат. Для `image` Core выставляет локальный `src` и alt-текст.

Поведение необязательных placeholders (required: false):

- Если placeholder не заполнен автором, Core **не вставляет содержимое** в соответствующий `data-placeholder` и при необходимости скрывает обёртку через атрибут `data-placeholder-hide` (шаблон обязан поддерживать этот атрибут для всех `required: false` блоков).
- Шаблон не должен рассчитывать на наличие значения в необязательном placeholder — верстка должна корректно отображаться при пустом блоке без видимых артефактов (пустые рамки, зазоры).
- Валидация и сохранение формы блокируются только при незаполненных **обязательных** полях (`required: true`). Незаполненные необязательные поля не блокируют сохранение.
- UI формы редактирования страницы показывает под необязательными полями hint: "Необязательно — если не заполнено, блок не отображается."

8.2.3 Совместимость при смене шаблона

Если страница создана в режиме `standard`, смена шаблона сохраняет содержимое и меняет только отображение.

Если страница создана в режиме `template`, она привязана к `templateKey` и схеме placeholders. При смене шаблона Core должен:

1. найти в новом шаблоне `contentTemplates[].key` с тем же `templateKey`;
2. если найден, применить новый layout к сохранённым values;
3. если не найден, показать автору состояние **"требуется сопоставление шаблона страницы"**;
4. позволить выбрать новый `contentTemplate` и вручную сопоставить совместимые placeholders;
5. не удалять старые values автоматически.

8.2.4 Стандартные и HTML-страницы

Режим `standard` использует канонические placeholders Core для переносимых страниц:

```
title
subtitle
media
body
primaryActionLabel
summaryFields
```

Режим `html` предназначен только для технических сценариев. HTML проходит санитизацию и отображается через `content.html` или `layouts.content`. Использование `html` должно быть явно видно в UI, потому что такая страница слабее стандартизирована, чем `template / standard`.

`content.intro` и `content.summary` — симметричные «закладки» раздела: `content.intro` рендерится перед вопросами раздела (`before_topic`), `content.summary` — после расчёта результата раздела (`after_topic`). Для `content.summary` Core дополнительно добавляет в публичный контекст готовый результат темы/раздела (`result.*` = результат РАЗДЕЛА, не всего теста — итог теста показывает макет `results`, §8.3). Шаблон может показать эти значения через placeholders типа `resultField`, но не рассчитывает их самостоятельно.

8.3 Макеты результатов

Макеты результатов могут использовать экранированные переменные и опциональные контролируемые слоты.

Опциональные слоты:

```
<div data-slot="results-summary"></div>
<div data-slot="result-variables"></div>
```

Пользовательские показатели результата остаются отдельной продуктовой функцией, но после вычисления публикуют значения в пространстве имён `result.*`.

Экран итогов несёт два уровня заголовков: зонтик «Ваш результат» и, под ним, до четырёх подблоков — сводка баллов, шкалы, показатели, темы, каждый со своим заголовком. Оба уровня надписаны надписями шаблона (§8.3.1), а состав и порядок подблоков — отдельной настройкой шаблона (§8.3.3). Та же иерархия и те же ключи действуют на адаптивных итогах, на итогах раздела и в отчёте.

Строка темы внутри подблока `topics` может нести ещё один уровень — строки разреза результата (подытоги по ключам оси, §10.6). Это необязательный блок внутри карточки: макет, который его не печатает, остаётся рабочим.

8.3.1 Надписи интерфейса: `labels[]`

Надпись — именованная строка интерфейса, которую объявляет ШАБЛОН с текстом по умолчанию, а автор теста может переформулировать или выключить (PRD-49). Заголовки блоков экрана итогов — «По шкалам», «Ваш результат» — до этого расширения были зашиты в макет: изменить формулировку или убрать заголовок, не убирая блок целиком, было нечем.

Надписи объявляются разделом ВЕРХНЕГО УРОВНЯ манифеста `labels[]`, а не полем внутри `contentTemplates[].settings[]`: одна надпись действует сразу на нескольких экранах (итоги, адаптивные итоги, итоги раздела, отчёт), тогда как `settings[]` по природе принадлежит одному варианту.

```
{
  "key": "recommendations.courses",
  "group": "Группы рекомендаций",
  "label": "Подпись группы курсов",
  "default": "Пройти обучение",
  "defaults": { "report": "Рекомендации по курсам" }
}
```

Поле	Обязательность	Смысл
<code>key</code>	да	Семантический ключ; по нему макет обращается к надписи (<code>labels.results.scales</code>) и по нему же строится путь дерева контекста (§10.5)
<code>group</code>	да	Группировка полей в редакторе — пятнадцать надписей сплошным списком нечитаемы
<code>label</code>	да	Подпись поля для автора
<code>default</code>	да	Текст по умолчанию, общий для всех экранов
<code>defaults.<экран></code>	нет	Умолчание конкретного экрана, когда оно отличается от общего

Экраны, для которых надпись может объявить собственное умолчание: `results`, `results.adaptive`, `section-results`, `report`. Приём «общее умолчание плюс умолчание экрана» здесь тот же, каким `resultsBlockOrder` (§8.3.3) объявляет состав и порядок подблоков — шаблон следует одной конвенции в обоих разделах.

Раздел необязателен: шаблон, не объявивший ни одной надписи, печатает свои жёсткие строки макета — подраздел «Заголовки и подписи» такому шаблону в редакторе не показывается.

Статические проверки при загрузке шаблона (`validateLabelDeclarations`, код `LABELS_INVALID`, см. §17.1):

- `key` обязателен и уникален внутри `labels[]` ;
- объявленная надпись обязана иметь `default` ;
- ключи не могут быть взаимными префиксами (`results` вместе с `results.heading`) — контекст резолвит путь как ДЕРЕВО (§10.5), а не как плоскую карту, и префиксный ключ не может одновременно быть строкой-листом и веткой с вложенными ключами.

8.3.2 Хранение значения и три состояния надписи

Значения автора хранятся ОТДЕЛЬНО от объявления: манифест несёт умолчания, тест — только отклонения от них.

- Общий словарь — `tests.design_settings_json.labels` : ключ отсутствует — печатается текст шаблона.
- Порядок подблоков — `tests.design_settings_json.resultsBlockOrder` (§8.3.3).
- Переопределения ОТЧЁТА — `tests.report_settings_json.labels` , той же формы. Отчёт может говорить иначе, чем экран итогов, если автор этого захотел; пусто — как на экране итогов.

Новых колонок не заводится: оба словаря лежат в уже существующих `jsonb` -полях теста.

Значение одной надписи — ЗАПИСЬ с двумя необязательными полями, а не голая строка:

```
{ "on": false }
{ "on": true, "text": "Профиль стилей" }
```

Запись, а не строка, потому что у надписи ТРИ различных состояния, и строкой их не развести: пустая строка неотличима от «поле никогда не открывали».

Состояние	Хранение	Что печатается
Не трогал	ключа в словаре нет	текст шаблона (<code>default</code> / <code>defaults.</code> <code><экран></code>)
Переформулировал	непустой <code>text</code> (<code>on</code> не <code>false</code>)	своя формулировка
Выключил	<code>{ "on": false }</code>	ничего — заголовок гасится, сам блок остаётся

Выключение — тумблер, а не очистка поля: у записи есть отдельный флаг `on` , поэтому «не трогал» и «выключил» различимы в данных, а не только по соглашению об пустой строке.

Разрешение идёт слоями поверх умолчания шаблона: словарь теста, затем (только для отчёта) переопределение отчёта. Слой либо ничего не говорит о ключе (пропуск — резолвер идёт глубже), либо гасит текст (`on: false` → пустая строка), либо задаёт свой текст. Одна функция на все хосты — `resolveLabels` (`shared/template/labels.ts`) — веб-хост, пакет SCORM и отчёт читают её ответ, ни один не считает надписи сам.

Перечень надписей ОТЧЁТА считается по макетам, а не объявляется. Редактор предлагает автору в настройках отчёта не все объявленные надписи, а только те, которые макеты вариантов отчёта действительно печатают: список собирается сканированием этих макетов на прямые пути `labels.<ключ>` (`readReportLabelKeys` , поле `reportLabelKeys` ответа `GET /api/templates/:id`) и отдаётся в порядке манифеста. Второй список в манифесте разошёлся бы с макетами молча, и автор продолжал бы включать заголовок, которого в документе нет. Для отчёта расчёт точен, а не приближен: состав и порядок подблоков (§8.3.3) документу не передаются, поэтому прямой путь `labels.<ключ>` — единственный способ попасть заголовку в PDF. Отсюда правило для разработчика шаблона: надпись, которую макет отчёта не печатает прямым путём, автору для отчёта не предложат. Шаблон, не объявивший ни одного варианта отчёта, берёт перечень «Стандартного» — тем же правилом, каким берёт его макеты (§8.4.4). Перечень считается на запрос, а не при регистрации шаблона: иначе он отставал бы от файлов шаблона до перерегистрации, как сам манифест.

8.3.3 Порядок и состав подблоков: `resultsBlockOrder`

Раздел манифеста верхнего уровня объявляет по каждому экрану, какие из пяти подблоков итогов (`summary` , `scales` , `indicators` , `topics` , `breakdown`) печатаются и в каком порядке.

```
"resultsBlockOrder": {
  "default": ["summary", "scales", "indicators", "topics", "breakdown"],
  "results.adaptive": ["topics", "scales", "indicators", "breakdown"]
}
```

Форма объявления — та же «общее умолчание плюс умолчание экрана», что и у надписей (§8.3.1): `default` отвечает за экраны, для которых нет отдельной записи, именованный ключ экрана (`results.adaptive`) переопределяет список целиком. Плоский массив вместо объекта — тоже валидная форма и равносителен единственному `default` : так шаблон, который экраны между собой не различает, объявляет один список на все.

Список экрана несёт СРАЗУ ДВА решения — состав и порядок. Ключа нет в списке экрана — на этом экране такого подблока не бывает: адаптивные итоги никогда не печатали сводку баллов, поэтому в их списке `summary` нет и не должно появляться. Порядок при этом читается позицией в массиве.

Раздел необязателен. Шаблон, не объявивший его, получает зашитое умолчание (`summary` , `scales` , `indicators` , `topics` , `breakdown`) — тот порядок, что макет печатал до появления этой настройки, плюс сводный разрез последним, — поэтому вид уже собранных тестов не меняется сверх переезда самих заголовков на зонтик и подзаголовки. Место `breakdown` в конце списка не декоративное: шаг 3 разрешения дописывает в конец всё, чего сохранённый порядок автора не упоминает, поэтому тест, собранный до появления подблока, всё равно получит его именно там — объявить его в константе иначе значило бы разойтись с тем, что печатает живой тест.

Настройка автора теста — ОДНА на все экраны, а не по экрану (§8.3.2): список шаблона одного экрана определяет, какие из выбранных автором ключей на этом экране применимы, и что делать с ключом, которого шаблон не знает. Разрешение (`resolveBlockOrder` , `shared/template/results-order.ts`):

1. для рендерящегося экрана берётся список шаблона — умолчание экрана либо общее (`templateBlockOrder`);
2. сохранённый порядок автора фильтруется по этому списку: ключ, которого список экрана не содержит, отбрасывается — экран не может напечатать то, для чего шаблон не выделил место;
3. ключ, который список экрана знает, а сохранённый порядок ещё не упоминает (шаблон обновился и добавил подблок позже, чем автор в последний раз сохранял порядок), дописывается в конец в порядке шаблона — новый подблок не выпадает и не ломает то, что автор уже расставил.

Ключ вне закрытого перечня (`summary` / `scales` / `indicators` / `topics` / `breakdown`) отбрасывается на любом слое без ошибки: раздел — данные, а не код, и обязан принимать значение, не роняя рендер.

8.4 Макеты отчёта о результатах

Отчёт — PDF-файл с итогами попытки, который обучающийся скачивает действием `download-report` (флаг `result.nav.showReport` на экране итогов, `state.canDownloadReport` на стартовом экране в кулдауне). Это не снимок экрана результатов, а самостоятельная страница со своей вёрсткой, и вёрстка эта принадлежит ШАБЛОНУ, а не ядру.

Страницу рисует тот же браузерный рендерер, что и ученические экраны; конвейер экспорта растеризует её и укладывает в A4. Ядро отвечает за данные и за файл, шаблон — за страницу. Данные, из которых страница собирается, — публичный контекст отчёта (§10.2).

8.4.1 Виды и объявление варианта

Видов два, и они РАЗНЫЕ, а не варианты одного: у них разное содержание, и вариант одного режима не может быть выбран для другого.

kind	Режим теста	Что печатает
report	обычный	баллы, проценты, вердикт «пройден / не пройден»
report.adaptive	адаптивный	подтверждённые уровни по темам, без баллов

Вид определяется РЕЖИМОМ теста; подмены вида выбором автора нет.

Вариант объявляется записью `manifest.contentTemplates[]` :

- `layoutFile` — ОБЯЗАТЕЛЕН. Общего макета «на весь вид» у отчёта нет (§5.3);
- `styleFile` — необязательная таблица стилей страницы (§8.4.3);

- `isDefault` — ровно один вариант на каждый объявленный вид;
- `settings[]` — поля, которые автор теста заполняет в настройках теста;
- `placeholders[]` — НЕ применяются: в отчёте нет содержимого, которое обучающийся читает как страницу. Объявленный непустой `placeholders[]` — ошибка валидации (§17.1).

Типы `settings[]` — общий закрытый перечень (§8.2.1) за вычетом `sequence`: последовательностей страниц у отчёта нет, поэтому такой тип отклоняется валидацией. Шаблон может объявить любое число вариантов каждого вида — например «с подложкой» и «строгий, под печать».

Назначение поля: `scope`

Поле варианта отчёта может объявить, к чему оно относится:

<code>scope</code>	Где автор его правит	Что это за поля
<code>content</code>	«Настройки» → блок обратной связи	что попадёт в документ: диаграмма по шкалам, её предел, состав блоков
<code>appearance</code> (умолчение)	«Оформление» → «Отчёт»	как документ выглядит: подложка, логотип, типографские параметры

Признак нужен потому, что автор ищет эти поля в разных местах: содержание документа — рядом с обратной связью, которую получит слушатель, облик — рядом с шаблоном и брендингом. Разложить поля по двум экранам может только тот, кто их объявил.

Умолчение — `appearance`, и оно не нейтральное, а совместимое: до появления признака все поля показывались в «Оформлении», и шаблон, который о признаке не знает, оставляет их ровно там, где автор их находил. Непонятное значение трактуется как `appearance` и шаблон не отклоняет.

8.4.1.1 Картинки отчёта — файлы шаблона

Подложка, логотип и любая декоративная графика отчёта принадлежат ШАБЛОНУ: он кладёт файлы в свой пакет, перечисляет их в `manifest.assets.images` и объявляет полем `settings[]` типа `image`, чей `default` — путь внутри шаблона. Ядро не знает ни имён этих полей, ни их файлов.

```
{
  "key": "backgroundImage",
  "type": "image",
  "label": "Подложка страницы",
  "default": "assets/report/bg.png"
}
```

Значение поля может быть двух видов, и разрешаются они по-разному:

Значение	Что это	Как разрешается
<code>assets/report/bg.png</code>	файл шаблона (умолчание манифеста либо путь, введённый автором)	против базы ХОСТА: <code>template/...</code> внутри пакета, <code>/api/templates/<id>/assets/...</code> на вебе
<code>{ "url": "/uploads/media/x.png" }</code> или абсолютный URL	картинка автора теста	берётся как есть

Перед растеризацией хост инлайнит эти значения в data-URL: растеризатор снимает то, что уже лежит в документе, и ничего не догружает — незагруженная подложка молча меняет PDF. Картинка, которую прочитывать не удалось, становится пустым значением, а не ошибкой: макет гейтит свои строки на значении и печатает страницу без неё.

Отсюда правило для макета: он читает `report.values.<key>`, а не какое-то отдельное поле контекста. Ядро не поставяет отчёту ни подложки, ни логотипа по умолчанию — их даёт шаблон.

8.4.2 Как выбор автора доходит до выдачи

Автор теста выбирает вариант и заполняет его поля; выбор — свойство ТЕСТА, а не шаблона и не темы (§16). Разрешение выбора против манифеста — ОДНА функция на все хосты (`shared/report/report-variants.ts` → `resolveReportBake`): иначе отчёт в LMS расходился бы с тем, что автор видел в предпросмотре.

Хост	Когда разрешается	Где лежит результат
SCORM-пакет	при сборке (манифест виден только сборщику)	<code>TEST_DATA.designSettings.report (layoutKey , styleFile , values)</code> ; CSS варианта вложен в <code>styles.css</code>
Веб	при запросе результата попытки	поле <code>reportRender</code> ответа <code>/api/attempts/:id/result</code> (макет, CSS, переменные темы, значения полей)
Предпросмотр в настройках теста	в браузере автора	из бандла шаблона, на несохранённом черновике

Значения полей — правки автора поверх `default` манифеста; поле, которого выбранный вариант не объявляет, отбрасывается, поэтому смена варианта не тащит чужие значения. При смене варианта переживают переход только ключи, объявленные обоими вариантами с ОДНИМ типом.

8.4.3 Среда стилей отчёта

Отчёт рендерится в служебном контейнере ГЛАВНОГО документа, вне сцены. Контракт задан явно, потому что среды хостов расходятся ровно в одном месте: дизайн-система у обоих лежит в главном документе, а CSS ШАБЛОНА в SCORM-пакете лежит в главном документе, тогда как на

вебе внедряется внутрь Shadow DOM экрана. Макет отчёта, опирающийся на `theme.css` или на слой сцены, выглядел бы верно в LMS и сломался бы в браузере.

Макету отчёта доступны:

1. **Компоненты дизайн-системы** (`.ou-*`) — как на любом экране.
2. **Токены темы и брендинг теста**. Хост записывает их CSS-переменными на КОНТЕЙНЕР отчёта (те же `buildTemplateCssVars` / `buildTemplateThemeCss`, что питают сцену, но с корневым селектором `.tb-report`). Работают `var(--ou-...)`, `hsl(var(--primary))`, `var(--font-sans)` и пинованная тема. Это ЕДИНСТВЕННЫЙ канал темы: `theme.css` целиком не подключается.
3. **Собственный styleFile варианта** — внедряется одинаково на обоих хостах.

Запрещены и отклоняются валидацией (§17.2):

- классы слоя сцены (`tb-scene*`): сцена описывает экран фиксированного вьюпорта, а отчёт — колонка шириной 595 px, печатаемая на A4;
- селекторы `:root`, `html`, `body` в `styleFile`: отчёт документом не является.

Запрет сделан исполнимым, а не пожеланием: корневой элемент макета обязан нести класс `tb-report`, а все селекторы `styleFile` — быть вложены в `.tb-report`.

Шрифты снимаются растеризатором из вычисленных стилей, поэтому шрифт, который должен попасть в PDF, обязан быть доступен в момент рендеринга (в пакете — из его ассетов, в вебе — из глобальных шрифтов приложения). Отсутствующий шрифт даёт не ошибку, а молча другой PDF — поэтому рендер отчёта входит в smoke-проверку (§17.3).

8.4.4 Деградация: шаблон без вида отчёта

Шаблон, не объявивший нужного вида, отчёта НЕ лишается. Страница собирается макетом одноимённого вида СТАНДАРТНОГО шаблона, а оформление (параметры брендинга, палитра, логотип) и значения полей берутся из АКТИВНОГО шаблона и настроек теста. Вместе с макетом из стандартного шаблона приезжает и его `styleFile` — иначе страница собралась бы без оформления.

Правило то же, что для системных экранов (§8.2), и работает на обоих хостах: в пакете — через резервные макеты, вложенные сборщиком, на вебе — повторным чтением из каталога `default`.

8.4.5 Кликабельные ссылки и вложение в пакет

Растровая страница сама по себе ссылок не несёт. Элемент макета с классом `pdf-link-btn` и атрибутом `data-url` конвейер экспорта превращает в НАСТОЯЩУЮ ссылку PDF по координатам элемента. Без этого контракта рекомендованный курс в отчёте останется картинкой.

Пакет несёт каталог шаблона целиком, поэтому макет и картинки варианта в нём уже есть — под `template/` (а при деградации, §8.4.4, — под `template-default/`). Сверх этого сборщик вкладывает CSS выбранного варианта в общий `styles.css` и запекает разрешённые пути картинок: к

моменту растеризации читать манифест и файлы из рантайма поздно. Пакеты, собранные до введения контракта, запечённого выбора не несут и собирают отчёт по каноническому виду — прежнее поведение.

На вебе картинки шаблона отдаются существующим роутом файлов шаблона (`GET /api/templates/:id/assets/*`), поэтому область сессии по ссылке-приглашению включает этот роут: без него ученик, пришедший по ссылке, получил бы отчёт без оформления.

8.4.6 Постраничная раскладка и принудительный разрыв

Отчёт — не одна длинная картинка: конвейер экспорта режет отрисованную страницу на листы А4. Разрезы раскладка ставит САМА — по высоте листа, по границам карточек и по нижним границам строк текста, — поэтому строка никогда не делится пополам. Решение это машинное: раскладка знает, что помещается на лист, но не знает, что документ читается разделами.

Приказать ей начать новый лист макет может пустым узлом с атрибутом `data-page-break` :

```
<section class="tb-report__card">...результаты по темам...</section>
<div data-page-break></div>
<section class="tb-report__card">...показатели...</section>
```

Контракт метки:

- **Атрибут, а не класс.** `data-page-break` стоит в одном ряду с `data-path` и `data-action` : это контракт РЕНДЕРЕРА, а не имя из стилевого пространства шаблона. Класс принадлежит шаблону, и внешний шаблон, переименовавший его у себя, потерял бы правило молча.
- **Стилей не требует.** Узел пуст и потому нулевой высоты, а разрез идёт по его ВЕРХНЕЙ границе: даже если шаблон случайно даст метке высоту, на бумаге она следа не оставит.
- **Любая глубина.** Метка действует прямым ребёнком корня отчёта, внутри карточки и внутри повтора `{{#each}}` — «каждое толкование показателя с новой страницы» пишется одной меткой в теле повтора. Внутри `{{#if}}` она печатается по тому же условию, что и остальная разметка, в том числе по значению поля варианта (§8.4.1), если разработчик шаблона отдаёт выбор автору.

Правила раскладки:

- принудительный разрыв СИЛЬНЕЕ всех прочих правил: и «карточку не рвём», и правило висячей строки, и предпочтение крупных границ отступают перед ним. Метка внутри карточки рвёт карточку вместе с её фоном и скруглениями — это осознанный выбор верстальщика;
- метка, над которой на текущем листе ничего нет (две подряд, метка в начале документа, метка сразу после другого разрыва), ИГНОРИРУЕТСЯ, как и метка после последнего содержимого: пустых листов механика не порождает;
- между двумя метками содержимое по-прежнему делится по высоте листа — раздел длиннее страницы режется обычным правилом, по нижним границам строк;

- **лист остаётся полноразмерным.** Разрыв меняет только то, где кончается СОДЕРЖИМОЕ; лист остаётся A4 с подложкой во всю высоту и ширину, полустраниц без фона не возникает.

Правило живёт в двух чистых модулях — `shared/report/paginate.ts` (сам разрез) и `shared/report/paginate-dom.ts` (измерение верхних границ меток в браузере), — которые зовут и конвейер PDF, и предпросмотр отчёта в редакторе. Поэтому скачанный файл, предпросмотр и отчёт внутри SCORM-пакета получают разрывы разом.

9. DSL браузерного рендера шаблонов

Браузерный рендерер поддерживает минимальный path-only DSL.

Поддерживается:

```
{{ path }}

{{#if path}}
  ...
{{/if}}

{{#unless path}}
  ...
{{/unless}}

{{#each path}}
  ...
{{/each}}

{{> partialName }}

{{& path }}
```

Не поддерживается:

- JavaScript;
- helper-функции;
- выражения внутри `if`;
- `{{{ path }}} — у контролируемого HTML одна форма записи, {{& path }}.`

Весь вывод `{{ path }}` экранируется как текст.

Контролируемый HTML: `{{& path }}`

`{{& path }}` печатает значение РАЗМЕТКОЙ, без экранирования. Это единственный канал разметки, доступный внутри `{{#each}} : data-placeholder` и `data-slot` адресуются по имени и повторяющемуся узлу не годятся, а именно в циклах печатаются толкования уровней и тексты рекомендаций, которым автор теста задал формат «Форматированный» или «HTML» (`feedback_json.format`, см. §10.4).

Что попадает в такие поля, решает ЯДРО, а не автор макета: контекст несёт готовую разметку в парном поле (`textHtml`, `textsHtml`), которую ядро строит из текста и его формата. Обычный текст ядро экранирует само и переводит переводы строк в `<br>`, поэтому `{{& ... }}` над полем `*html` безопасен ровно в той же мере, в какой безопасен `richText / html` контентной страницы: источник один и тот же — автор теста.

Применять `{{& ... }}` к произвольному полю контекста НЕЛЬЗЯ: поля, не построенные ядром как разметка, печатаются через `{{ ... }}`.

HTML/rich content страницы по-прежнему вставляется через контролируемые placeholders/слоты, например:

```
data-placeholder="body"
data-slot="content-body"
data-slot="question-text"
data-slot="question-interaction"
data-slot="question-feedback"
```

Для контентных страниц предпочтителен `data-placeholder`, потому что он связан со схемой `contentTemplates[].placeholders[]`. `data-slot="content-body"` остаётся допустимым fallback для режима `html` и старых общих content layouts.

9.1 Контекст `each`

Внутри `each` текущий элемент становится текущим контекстом:

```
{{#each sections}}
  <button>{{ title }}</button>
{{/each}}
```

Корневой контекст доступен как:

```
{{ @root.test.title }}
```

Мета-переменные цикла:

@index	индекс с нуля
@number	индекс с единицы
@first	boolean
@last	boolean

9.2 Частичные шаблоны

Частичные шаблоны объявляются в `manifest.partials`:

```
{
  "partials": {
    "topicNav": "partials/topic-nav.html",
    "progress": "partials/progress.html"
  }
}
```

Использование:

```
{{> topicNav }}
```

Частичные шаблоны получают тот же публичный контекст, что и текущий макет.

Если частичный шаблон `topicNav` отсутствует, Core может использовать стандартный рендерер навигации по темам как резервный вариант.

10. Публичный контекст рендера

Все макеты получают общий публичный контекст.

Типизированный контракт контекста рендера — `PublicRenderContext` с неймспейсами `course`, `state`, `result`, `retake`, `transition`, `design`, `review`, `sectionResult`, `sectionIntro`; страница отчёта получает сверх того неймспейс `report` (§10.2). Именно против него резолвятся `{{ path }}` и `data-path` в макетах (`data-path="course.title"`, `{{ result.scorePercent }}`, `{{#if state.canResume}}`). Перечень полей каждого неймспейса — в §7 и в разделах ниже. Пути `progress.*` доступны `resultField`-плейсхолдерам только в SCORM-рантайме (резолв против внутреннего `TEST_DATA`); на веб-хосте `resultField` резолвится против `PublicRenderContext`, где `progress.*` как топ-уровневого пути нет (есть `result.*` и `sectionResult.*`).

Контентная страница дополнительно получает неймспейс `page.*`, вычисленный ядром из структуры теста: `page.dots` — точки индикатора последовательности (по одной на страницу отрезка, у текущей выставлен признак активной), `page.dotIndex` и `page.dotsTotal` — позиция и размер отрезка, `page.canGoBack` — доступен ли возврат на предыдущий экран в текущем режиме прохождения, `page.nextLabel` — подпись кнопки «вперёд» (настройка `nextLabel` страницы, по умолчанию «Далее»). Страница вне последовательности получает пустой набор точек, поэтому макет с индикатором деградирует до простой страницы. Рисовать индикатор и кнопку возврата — решение МАКЕТА; данные ядро отдаёт всегда. `page.dots/dotIndex/dotsTotal/canGoBack` автор не вводит; `page.nextLabel` он задаёт настройкой варианта. Все три хоста (SCORM-пакет, веб-хост, предпросмотр) читают `page.*` из одного общего ядра (`shared/template/page-sequences.ts`), так что считают одинаково.

Пример ранней (нереализованной) формы контекста:

```
{
  "test": {
    "id": "test-1",
    "title": "Сертификация",
    "description": "",
    "navigationPolicy": "linear"
  },
  "page": {
    "id": "q-1",
    "type": "question",
    "kind": "question.single",
    "title": "Вопрос 1",
    "question": {
      "id": "q-1",
      "type": "single",
      "media": null
    },
    "answerState": {
      "hasAnswer": false,
      "locked": false,
      "feedbackVisible": false
    },
    "feedback": null
  },
  "sections": [
    {
      "id": "topic-1",
      "title": "Тема 1",
      "isActive": true,
      "isPassed": false,
      "className": "is-active"
    }
  ],
  "progress": {
    "active": {
      "current": 3,
      "total": 20,
      "percent": 15
    },
    "question": {
      "current": 3,
      "total": 20,
```

```

    "percent": 15
  },
  "page": {
    "current": 6,
    "total": 28,
    "percent": 21
  }
},
"nav": {
  "mode": "linear",
  "canPrev": true,
  "canNext": false,
  "canSubmitAnswer": true,
  "canFinish": false,
  "nextLabel": "Далее",
  "submitAnswerLabel": "Принять ответ",
  "finishLabel": "Завершить тест",
  "nextClassName": "is-disabled"
},
"params": {
  "brand.primaryColor": "#0066cc",
  "progress.mode": "questions"
},
"assets": {},
"runtime": {
  "templateApiVersion": "1.0"
}
}

```

Core подготавливает готовые классы состояния (например `passClass` , `statusLabel`); DSL шаблона классы через выражения не вычисляет.

10.1 Обратная связь и правильные ответы

Core раскрывает данные правильных ответов в публичном контексте только тогда, когда текущее состояние Core и настройки теста это разрешают.

До обратной связи:

```
{
  "answerState": {
    "hasAnswer": false,
    "locked": false,
    "feedbackVisible": false
  },
  "feedback": null
}
```

После отправки ответа, если правильные ответы можно показывать:

```
{
  "answerState": {
    "locked": true,
    "feedbackVisible": true,
    "scoreRatio": 0.5,
    "status": "partial"
  },
  "feedback": {
    "text": "Частично правильно",
    "correctAnswerPublic": {}
  }
}
```

Шаблоны не получают поддерживаемого прямого доступа к внутреннему `TEST_DATA`.

10.2 Контекст отчёта: `report.*`

Страница отчёта (§8.4) получает `course.*`, `design.*` и `result.*` в ТОЧНО ТОМ ЖЕ виде, что экран результатов, плюс собственный неймспейс `report.*`. Контракт публичный и расширяется только аддитивно: на него опираются макеты внешних шаблонов.

`result.*` строится ТЕМ ЖЕ строителем, что и экран результатов. Отчёт не вправе показать иной вердикт, чем экран, с которого его скачали, и не считает ничего самостоятельно — два независимых расчёта одного вердикта всегда расходятся.

Поле	Описание
<code>attemptDateLabel</code>	Готовая подпись даты прохождения (<code>дд.мм.гггг чч:мм</code>)
<code>attemptsCountLabel</code>	Готовая подпись числа попыток, со склонением; ПУСТА у теста, который ничего не оценивает (макет гейтит строку)
<code>learnerName / hasLearnerName</code>	ФИО слушателя (<code>cmi.learner_name</code> в LMS, пользователь сессии в вебе) и гейт строки: имя может быть неизвестно
<code>gridColumns</code>	Число колонок сетки тем, вычисленное ядром
<code>verdictHeadline / verdictBadge / verdictClass</code>	Заголовок, подпись бейджа и класс вердикта (<code>is-pass</code> / <code>is-fail</code>)
<code>correctLabel / earnedPointsLabel</code>	Готовые подписи «верно из общего числа» и заработанных баллов
<code>ringDasharray / ringDashoffset</code>	Геометрия дуги отчёта (своя, отличная от кольца экрана итогов)
<code>hasTopics</code>	Гейт блока тем
<code>courses[] / hasCourses , events[] / hasEvents</code>	Рекомендованные материалы и мероприятия: в обычном режиме — по проваленным темам без дублей, в адаптивном — по всем темам, у которых они есть, с названием темы
<code>isPreview</code>	Признак предпросмотра — макет может показать пометку «образец»
<code>values.<key></code>	Значения <code>settings[]</code> варианта (§8.4.1). Поля типа <code>image</code> приходят готовой строкой: путь файла шаблона разрешён против базы хоста и инлайн в data-URL (§8.4.1.1). Пустая строка = картинки нет, макет гейтит строку на ней

Строки `result.topicResults[]` в отчёте несут дополнительные ГОТОВЫЕ подписи, которых на экране нет: в обычном режиме — `verdictLabel` , `barPercent` , `countsLabel` , `pointsFixedLabel` ; в адаптивном — `hasCounts` , `answeredLabel` , `correctLabel` , `achievedClass` . Строки разреза (`breakdown[]` , §10.6) приходят в отчёт в том же виде, что и на экран: настройка показа одна.

Текста обратной связи темы среди них НЕТ: он печатается один раз, в консолидированном блоке `result.recommendations` . Прежний признак `showFeedback` снят вместе со слотом в карточке темы. Исключение — адаптивный отчёт: там `feedback / hasFeedback` строки темы означают обратную связь ДОСТИГНУТОГО УРОВНЯ (либо текст провала темы), в консолидированный блок она не подаётся, и слот в карточке сохранён.

Правило §10 действует и здесь: DSL не считает. Проценты, смещение дуги, число колонок, склонения и даты приходят готовыми.

10.3 Вводный блок: `result.introHtml`

Авторский текст, который идёт ПЕРВЫМ — до сводки баллов, результатов по темам, измерений и рекомендаций. Объясняет слушателю, что он сейчас читает.

Приходит РАЗМЕТКОЙ, уже построенной ядром из текста и его формата (§10.4), поэтому макет печатает его через `{{& ... }}`:

```
{{#if result.introHtml}}<div class="tb-intro">{{& result.introHtml }}</div>{{/if}}
```

Пусто или поля нет = блока нет; второго признака у него не заведено. У экрана итогов и у отчёта тексты РАЗНЫЕ (их задают отдельно), но имя поля одно: макет печатает «свой вводный блок», не зная, чей контекст ему дали.

10.4 Авторский текст и его формат

Тексты, которые пишет автор ТЕСТА — обратная связь теста, толкование уровня шкалы, исход показателя, — приходят в контекст ПАРАМИ: строка и её разметка.

Поле	Что несёт
<code>result.recommendations.texts[]</code>	Тексты консолидированного блока строками
<code>result.recommendations.textsHtml[]</code>	Они же разметкой, индекс в индекс
<code>result.scales[].text</code> / <code>result.indicators[].text</code>	Толкование уровня строкой
<code>result.scales[].textHtml</code> / <code>result.indicators[].textHtml</code>	Оно же разметкой

Разметку строит ядро из текста и его формата (`feedback_json.format` : `plain` , `richText` , `html`). Обычный текст экранируется, а переводы строк становятся `<br>` — абзацы автора доходят до слушателя; `richText` и `html` печатаются как есть. Политика та же, что у полей контентной страницы (§8.2.1), и источник тот же — автор теста.

Макет печатает поле `*Html` через `{{& ... }}` (§9), а парную строку — через `{{ ... }}`. Шаблон, который печатает только строку, остаётся рабочим: он покажет текст без форматирования, как показывал всегда.

Гейт блока стоит на СТРОКЕ (`{{#if text}}`), а не на разметке: пустая строка и пустая разметка появляются вместе, но признаком наличия текста остаётся сам текст.

10.5 Надписи итогов и состав подблоков: `labels.*`, `result.blocks`

Экран итогов, адаптивные итоги, итоги раздела и отчёт получают в контексте дерево `labels.*` — уже РАЗРЕШЁННЫЕ тексты всех надписей, объявленных шаблоном (§8.3.1), в форме, которую адресует DSL. Пустая строка означает «не печатать»: макет гейтит блок на самом значении надписи, а не на отдельном флаге видимости.

```
{{#if labels.results.scales}}<h3 class="tb-scene__subhead">{{ labels.results.scales }}</h3>
{{/if}}
```

Дерево строит `labelsTree` (`shared/template/labels.ts`) из плоской карты «ключ → текст», которую отдаёт `resolveLabels`: путь `results.scales` разбивается по точкам, потому что `{{ labels.results.scales }}` резолвится обходом вложенных объектов, а не поиском по буквальной строке ключа. Ключ, который шаблон не объявил или который конфликтует с другим (взаимный префикс, отклоняется валидацией §17.1, но старый пакет мог быть собран до её введения), в дерево не попадает — макет, обратившийся к нему, получает пустую строку. Правило то же, что и везде в этом разделе: сборка контекста ТОТАЛЬНА, экран ученика не имеет права упасть из-за объявления шаблона.

`result.blocks[]` — состав и порядок подблоков итогов (§8.3.3), уже отобранных и упорядоченных ядром. Каждый элемент несёт `key`, эффективный `heading` (пустая строка — заголовок выключен, сам блок при этом остаётся) и ОДИН булев флаг вида (`isSummary` / `isScales` / `isIndicators` / `isTopics` / `isBreakdown`) — DSL не умеет сравнивать строки (§9), поэтому выбор ветки шаблону нужен уже готовым. Макет обходит массив ОДНИМ циклом вместо пяти фиксированных секций:

```
{{#each result.blocks}}
  {{#if heading}}<h3 class="tb-scene__subhead">{{ heading }}</h3>{{/if}}
  {{#if isScales}} ... {{/if}}
{{/each}}
```

Видимость подблока (входит он в `result.blocks` вообще) и его заголовок (пуст ли `heading`) — независимые решения: скрытый подблок уносит заголовок с собой, а погашенный заголовок подблок не скрывает. Зонтичный заголовок (`labels.results.heading`) печатается только тогда, когда `result.blocks` непусто — видимых подблоков нет, печатать нечего.

10.6 Разрез результата: `result.topicResults[].breakdown`

Разрез — подытог попытки по ключу оси в пределах области (PRD-50). Ось — способ извлечь из выданного вопроса ключи; сейчас зарегистрирована одна, `tag` (подтемы вопроса). Область строки темы — РАЗДЕЛ ВЫДАЧИ: один ключ, заведённый в двух разделах, даёт в каждом свою запись.

Строка темы получает необязательный массив `breakdown[]` — уже подготовленные ядром строки:

Поле	Описание
<code>key</code>	Ключ оси — то, что печатается подписью строки (название подтемы)
<code>items</code> / <code>answered</code>	Сколько выданных вопросов несут ключ и сколько из них отвечено
<code>earned</code> / <code>possible</code>	Заработанные и возможные баллы по ключу, округлены до одного знака
<code>percent</code>	Величина ВЫБРАННОЙ автором базы, один знак после запятой — неокруглённый двойник <code>barPercent</code> . Макет, которому нужно число, печатает его и может не знать, какая база в силе
<code>percentUnits</code> / <code>percentPoints</code>	Нормированная доля (вес вопроса = 1) и балльная доля, по одному знаку. Балльная — валюта вердикта
<code>barPercent</code>	Ширина полосы в процентах, ОКРУГЛЕНА до целого; макет подставляет её в стиль
<code>showValue</code>	Печатать ли число рядом с полосой — гейт этого элемента разметки
<code>valueLabel</code>	Готовая подпись значения («50 %»); пуста при <code>showValue: false</code>
<code>passed</code>	Вердикт ключа: <code>true</code> / <code>false</code> / <code>null</code> . <code>null</code> — порога у ключа нет либо ключ не попал в выданное, и строка НЕ утверждает ничего
<code>passClass</code>	Готовый класс вердикта (<code>is-pass</code> / <code>is-fail</code> / пусто)
<code>statusLabel</code>	Готовая подпись вердикта («Пройдено» / «Не пройдено» / пусто)

Числа лежат в строке С САМОГО НАЧАЛА, даже когда поставляемые шаблоны печатают одну полосу: сторонний шаблон не может показать «верно 4 из 7», если чисел в контексте нет, а добавление поля позже — правка контракта, которую придётся возить по обоим хостам и по всем уже собранным пакетам. Форматирование при этом готовит ЯДРО (`showValue` , `valueLabel`), чтобы раскладка не решала его сама. Вердикт ключа приехал в строку тремя полями (`passed` , `passClass` , `statusLabel`), а не одним, по той же причине, по какой тремя приезжает вердикт темы: макет не должен ни считать его, ни переводить. Порог задаёт автор теста ключу раздела; ключ без порога и ключ, не попавший в выданное, дают `passed: null` и пустые класс с подписью, поэтому строка никогда не объявляет провал, которого гейт не выносил.

```

{{#if breakdown}}
<div class="tb-topic__breakdown">
  {{#each breakdown}}
    <div class="tb-breakdown__row" data-item="{{ key }}">
      <div class="tb-breakdown__name">{{ key }}</div>
      <div class="tb-breakdown__bar"><span style="width: {{ barPercent }}%;"></span></div>
      {{#if showValue}}<div class="tb-breakdown__val">{{ valueLabel }}</div>{{/if}}
    </div>
  {{/each}}
</div>
{{/if}}

```

Правила контракта:

- **Поля нет вовсе**, когда автор теста показ не включил или в теме нет ни одного ключа. Пустой массив в DSL ложен, поэтому один гейт `{{#if breakdown}}` убирает блок целиком вместе с его заголовком — второго признака видимости не заведено.
- **Показ — настройка ТЕСТА, а не оформления**: три положения (не показывать / полоса / полоса и число) и база показа (доля вопросов либо доля баллов). Настройка одна на все поверхности: экран итогов и отчёт печатают одно и то же.
- **База показа не меняет вердикт**. Порог ключа, когда он задан, всегда сравнивается с БАЛЛЬНОЙ долей — той же валютой, что и порог раздела; переключение вида отображения меняет только число и длину полосы. Поэтому ни `percent`, ни баллы строкой не передаются: макет печатает то, что ядро уже выбрало.
- **Это список ключей, а не разложение темы на части**. Ключи не обязаны разбивать выборку: вопрос может нести несколько ключей или ни одного, и сумма строк не обязана сходиться с итогом темы. Подписывать блок как «состав раздела» нельзя.
- **Те же строки в отчёте**. `result.*` в отчёте строится тем же построителем (§10.2), поэтому `breakdown[]` приходит и туда; макет отчёта печатает его своей вёрсткой.
- Считает разрезы один общий модуль (`shared/breakdown/`), который едет в бандл `shared-runtime`, поэтому веб-хост и пакет SCORM печатают на одной попытке одно и то же.

Имена классов в примере принадлежат «Стандартному» шаблону; контрактом является только состав данных.

10.7 Блоки разделов: `result.topicGroups`, `result.ungroupedTopics`

Автор теста может собрать разделы в НАЗВАННЫЕ блоки — «Обязательная часть», «Дополнительная часть» — и экран итогов печатает карточки тем по блокам, каждый со своим счётчиком «пройдено N из M». Вложенность ровно одна: тест держит блоки, блок держит разделы, дерева нет.

Поле блока	Описание
<code>key</code>	Ключ блока, каким его завёл автор
<code>label</code>	Заголовок блока; может быть пустым, если автор его не заполнил
<code>topics</code>	Карточки тем блока в порядке выдачи — ТЕ ЖЕ объекты, что и в <code>topicResults</code>
<code>passedCount</code> / <code>totalCount</code>	Разделы блока с вердиктом «пройден» и разделы с любым ВЫНЕСЕННЫМ вердиктом (раздел без вердикта не считается)
<code>counterLabel</code>	Готовый счётчик («1 / 2»); макет ничего не вычисляет

Правила контракта:

- **Поля нет вовсе**, когда автор блоков не заводил или ни один раздел в них не попал. Шаблон, ничего не знающий о блоках, печатает плоский `result.topicResults`, как печатал.
- **`topicResults` остаётся ПОЛНЫМ** рядом с блоками — и сгруппированные карточки, и несгруппированные. Поэтому макет, печатающий блоки, гейтитса на `topicGroups` и печатает `topicGroups` + `ungroupedTopics` ВМЕСТО плоского списка, иначе тема выйдет дважды.
- **`ungroupedTopics`** — карточки, не попавшие ни в один блок, в своём порядке; печатаются ПОСЛЕ всех блоков. Поле едет отдельно потому, что DSL не умеет фильтровать список: без него макет не отличил бы уже напечатанные карточки от оставшихся. Поля нет, когда блок нашёлся каждой карточке.

10.8 Сводный разрез по тесту: `result.breakdown`

Разрез из §10.6 живёт в пределах РАЗДЕЛА. Тот же ключ, заведённый в двух разделах, даёт две строки, и сложить их макет не вправе. Сводный блок отвечает на другой вопрос — «как ключ выглядит по всему тесту» — и приезжает отдельным подблоком итогов (`breakdown` в `result.blocks`, §8.3.3), одной строкой на ключ.

Строки — того же типа, что и в карточке темы (таблица §10.6), поэтому вёрстка блока повторяет вёрстку строки. Считает их ядро отдельным проходом по выданному (`shared/breakdown/`), а не суммированием строк тем: сумма дала бы другое — и неверное — число, ради чего блок и заведён.

```

{{#if isBreakdown}}
<div class="tb-breakdown">
  {{#each @root.result.breakdown}}
    <div class="tb-breakdown__row {{ passClass }}" data-item="{{ key }}">
      <div class="tb-breakdown__name">{{ key }}{{#if statusLabel}}<span class="tb-
breakdown__status">{{ statusLabel }}</span>{{/if}}</div>
      <div class="tb-breakdown__bar"><span style="width: {{ barPercent }}%;"></span></div>
      {{#if showValue}}<div class="tb-breakdown__val">{{ valueLabel }}</div>{{/if}}
    </div>
  {{/each}}
</div>
{{/if}}

```

Правила контракта:

- **Где печатать — выбор автора теста**, третьим полем настройки показа: в карточках тем, сводным блоком или и там, и там. Поля нет в настройке, сохранённой раньше, — она читается как «в карточках тем», то есть ровно то, что тест печатал до появления блока.
- **Поля нет вовсе**, когда блок не выбран или попытка не дала записей области теста.
- **Заголовок блока** — надпись `results.breakdown` (§8.3.1), умолчание «Разрез результата»; позиция блока — `resultsBlockOrder` (§8.3.3), в поставляемом порядке он последний.
- **Адаптивный режим печатает ТОЛЬКО сводный блок**. Карточка темы там говорит подтверждённым уровнем и полос не несёт (§10.6 к ней неприменим), а сводный блок считается по тому же выданному и печатается и на экране, и в отчёте.
- **Те же строки в отчёте** — `result.*` отчёта строит тот же построитель (§10.2).

11. Навигация

Поддерживаемые режимы навигации:

```

linear
free
locked

```

Шаблон объявляет поддерживаемые режимы в `manifest.capabilities.navigation`.

Бизнес-политикой навигации владеет тест. Возможности шаблона только ограничивают доступные режимы.

Эффективный режим навигации:

```
effective mode = test navigationPolicy, ограниченная возможностями шаблона
```

Параметры шаблона могут управлять представлением навигации, но не должны ослаблять политику теста.

Пример:

```
{
  "test": {
    "navigationPolicy": "linear"
  },
  "params": {
    "navigation.presentation": "sidebar"
  }
}
```

12. Runtime API для `template.js`

`template.js` опционален и исполняется в браузере обучающегося.

Реализованный API:

```
TestBuilder.template.on(event, handler);    // события жизненного цикла
TestBuilder.template.emit(event, data);
TestBuilder.context.get();                  // { params } – эффективные значения параметров
TestBuilder.scorm.commit();                 // фиксация SCORM-состояния
TestBuilder.ui.toast(message);              // заглушки: только console.warn
TestBuilder.ui.modal(options);
TestBuilder.renderers.register(id, fn);     // рендерер для resultField
TestBuilder.renderers.render(field, context, allowed);
```

Реально эмитируется одно событие — `page:enter` (на контентных страницах).

Вне охвата (логика теста задаётся сценарием, см. §14): `vars.*` (переменные времени прохождения), `nav.*` (программная навигация), `timer.*`, `ui.setState`, а также `scorm.setSuspendData` / `scorm.addInteraction`.

Прямой доступ к `window.API_1484_11` не входит в поддерживаемый контракт. Поддерживаемые SCORM-операции идут через `TestBuilder.scorm`.

13. Переменные времени прохождения — вне формата

Изменяемые переменные времени прохождения и их сохранение в `suspend_data` в формат шаблона не входят. Тест не оперирует произвольным мутабельным состоянием: логика прохождения задаётся декларативным сценарием (`flowPolicy` , см. §14), а вычисляемые результаты — это шкалы (`scale.*`) и показатели (`result.*`), которые Core публикует сам после расчёта результата. Шаблону в контексте рендера доступно пространство `result.*` с показателями (§10); собственного мутабельного состояния у шаблона нет.

14. Логика прохождения — вне формата

Логика прохождения теста задаётся декларативным сценарием, а не шаблоном и не универсальным движком правил. Ветвление и завершение выражаются структурными настройками сценария — `flowPolicy` (`flowMode` , `completionPolicy` , `sectionUnlockRules` , `passDecisionPolicy`), а условия по результату — показателями и адаптивными уровнями. Шаблон на эту логику не влияет: он отрисовывает экраны, которые ему передаёт Core.

15. Реестр шаблонов

Для встроенных и загруженных шаблонов используется один API реестра.

Концептуальные записи:

```
{
  "id": "corporate",
  "sourceType": "builtin",
  "sourcePath": "server/scorm/templates/corporate"
}
```

```
{
  "id": "rtk-custom",
  "sourceType": "uploaded",
  "sourcePath": "uploads/templates/rtk-custom"
}
```

Core использует:

```
TemplateRegistry.getTemplate(id)
```

и получает одну и ту же нормализованную файловую структуру независимо от физического источника.

16. Настройки дизайна теста

Рекомендуемая форма хранения:

```
{
  "templateId": "corporate",
  "templateVersion": "1.2.0",
  "templateApiVersion": "1.0",
  "params": {
    "brand.primaryColor": "#0066cc",
    "progress.mode": "questions"
  }
}
```

Строгое закрепление файловой версии не требуется. Сохранённые версии используются для диагностики, проверки совместимости и сообщений о миграции параметров.

Выбор варианта ОТЧЁТА и значения его полей (§8.4) хранятся ОТДЕЛЬНО от настроек дизайна:

```
{
  "standard": {
    "variantKey": "report.standard",
    "values": { "backgroundImage": { "url": "/uploads/media/bg.png" } }
  },
  "adaptive": {
    "variantKey": "report.adaptive.compact",
    "values": { "headline": "Итоги аттестации" }
  }
}
```

Ключи ветвей — РЕЖИМЫ теста, а не виды манифеста: тест одного режима заполняет одну ветку, но обе сохраняются, чтобы настройка не терялась при смене режима. Отсутствие ветки означает вариант `isDefault` активного шаблона со значениями `default` из манифеста, а отсутствие вида — деградацию по §8.4.4.

Тем же правилом настройка показа разреза (§10.6) — своя колонка теста, а не запись в настройках дизайна: она решает, что ученик увидит, а не как это выглядит, и правится на вкладке «Настройки».

Разнесение сделано намеренно, а не по недосмотру: настройки дизайна коммитятся черновиком вкладки «Оформление», тогда как поля отчёта автор правит в блоке обратной связи вкладки «Настройки». Общее хранилище связало бы две вкладки порядком сохранения. Следствие для реализации: смена шаблона может обнулить набор доступных вариантов, поэтому доступность пересчитывается на ЧЕРНОВОМ `templateId` и автор предупреждается до сохранения.

Реализация. Настройки дизайна — `tests.design_settings_json`, выбор отчёта — `tests.report_settings_json` (обе колонки nullable). Публикация теста пиннит выбранный вариант и значения полей в снапшот вместе с остальным содержимым: отчёт по старой попытке собирается тем макетом и теми параметрами, что действовали на момент выдачи.

17. Валидация

17.1 Структурная валидация

Блокирующие ошибки:

Блокируют только нарушения целостности пакета и безопасности — то, при чём шаблон нечитаем или небезопасен и подставить вместо него нечего:

- невалидный ZIP;
- отсутствующий или невалидный `manifest.json`;
- отсутствующие обязательные поля манифеста;
- неподдерживаемый `templateApiVersion`;
- отсутствующие файлы, на которые ссылается манифест;
- отсутствующий или невалидный `assets.preview`;
- отсутствующий или невалидный `preview`;
- отсутствующий, невалидный или несовместимый `preview.demoData`;
- внешние URL в ресурсах/макетах/скриптах/стилях, на которые ссылается манифест;
- невалидный DSL в макете (незакрытый блок, `{{{ }}`, выражение) — код `LAYOUT_TEMPLATE_SYNTAX`;
- невалидные `contentTemplates[]`: отсутствует `key`, `label` или `kind` (по схеме манифеста);
- неизвестный тип поля в `placeholders[]` или `settings[]`, а также несовместимые атрибуты типа (например `select` без списка вариантов);
- невалидные объявления вариантов отчёта (§8.4): вариант без `key` или без `layoutFile`, отсутствующий в пакете файл макета или `styleFile`, тип `sequence` либо непустой `placeholders[]` у варианта отчёта, не ровно один `isDefault` на объявленный вид;
- нарушение среды стилей отчёта (§8.4.3): макет без корневого класса `tb-report`, классы слоя сцены в макете отчёта, селектор `styleFile` вне `.tb-report` либо адресующий документ (`:root / html / body`);
- невалидное объявление надписей `labels[]` (§8.3.1): не массив, запись без `key`, дублирующийся `key`, надпись без `default`, взаимно-префиксные ключи;

- невалидные возможности (`capabilities`).

Предупреждения:

- неиспользуемые параметры;
- неиспользуемые ресурсы;
- отсутствующий макет — экран рендерится из стандартного шаблона (§8.2);
- отсутствующий слот, включая слоты вопросов и `page-content` ;
- отсутствующие хуки/действия `shell.html` ;
- `route` в `preview.routes` , для которого нет `layout/template/capability` ;
- предупреждения консоли в браузерной smoke-проверке;
- `fallback renderer` в `live preview` , если страница осталась работоспособной;
- отсутствующее покрытие необязательного `route` в `demo dataset` ;
- возможность объявлена, но опциональный `partial` отсутствует там, где существует резервный рендерер `Core` .

Реализация. Фактические блокирующие коды структурного валидатора (`server/services/template-validation.ts`): `ZIP_TOO_LARGE` , `MANIFEST_MISSING` , `MANIFEST_INVALID_JSON` , `MANIFEST_SCHEMA` , `ID_PATTERN` , `API_VERSION_UNSUPPORTED` , `ID_EXISTS` , `ID_MISMATCH` , `REQUIRED_FIELD_MISSING` (только `assets.preview / preview / params / capabilities`), `FILE_MISSING` , `LAYOUT_TEMPLATE_SYNTAX` , `EXTERNAL_URL` , `DEMADATA_INVALID_JSON` , `REPORT_VARIANT_INVALID` (сообщение называет вариант и место замечания), `LABELS_INVALID` (сообщение называет индекс и ключ; функция — `validateLabelDeclarations` , §8.3.1). Проверки отчёта идут дважды и с разной глубиной: при ЗАГРУЗКЕ доступны файлы пакета, поэтому проверяются макет и CSS; активационный гейт файлов не открывает и проверяет только ОБЪЯВЛЕНИЕ (та же схема, что у проверки тем). Предупреждения: `SHELL_CONTRACT` , `QUESTION_CONTRACT` , `CONTENT_CONTRACT` (нет `data-slot="page-content"`), `OPTIONAL_LAYOUT_MISSING` (не объявлен макет экрана), `UNUSED_FILE` . Определения `params` и объявления `systemPages` схемно НЕ валидируются (проходят через `passthrough`); их наличие в списках выше — часть контракта.

Проверка типов полей выполняется при ЗАГРУЗКЕ и при АКТИВАЦИИ шаблона: невалидный шаблон не принимается и не активируется, сообщение содержит ключ варианта и ключ поля. Ранее установленные шаблоны продолжают работать, но помечаются невалидными с указанием причины, а страница с полем неизвестного типа показывает в редакторе явную диагностику вместо подмены контроля.

17.2 Валидация макетов

Слоты, которые проверяет структурный валидатор (согласовано с §7 и §8.1). **Ни один из них не блокирует активацию:** их отсутствие даёт предупреждение и `fallback` экрана на стандартный шаблон (§17.1). Объявлять их всё равно следует — иначе шаблон отдаёт свой экран стандартному и теряет оформление.

Оболочка `shell.html` — область страницы (иначе `SHELL_CONTRACT`):

```
<div data-slot="page"></div>
```

Макет вопроса — два слота (иначе `QUESTION_CONTRACT`):

```
<div data-slot="question-text"></div>
<div data-slot="question-interaction"></div>
```

Контентная страница — слот `page-content` (иначе `CONTENT_CONTRACT`):

```
<div data-slot="page-content"></div>
```

Кнопки `data-nav="next"` / `data-action="answer-submit"` / `data-action="test-finish"` валидатор в оболочке **не требует** (Core привязывает их делегированием, когда они есть) — это относится к перспективным расширенным интерактивам (§3). Дополнительно валидатор компилирует каждый макет: невалидный DSL (незакрытый блок, `{{{ }}`, выражение) даёт блокирующую ошибку `LAYOUT_TEMPLATE_SYNTAX` — единственная блокирующая проверка макета ЭКРАНА.

Макет ОТЧЁТА проверяется строже, и эти проверки блокирующие (§17.1):

```
<div class="tb-report"> ... </div>          <!-- корневой класс обязателен -->
```

```
.tb-report__card { ... }                    /* допустимо: селектор скоуплен */
:root { --brand: #000; }                    /* запрещено: адресует документ */
.tb-scene__footer { ... }                   /* запрещено: слой сцены */
```

Строгость здесь не про вкус: несоответствие §8.4.3 в SCORM-пакете не проявляется и всплывает только в браузере, то есть на живой приёмке. Ревью такое не ловит — ловит только проверка.

17.3 Браузерная smoke-проверка

После структурной валидации запускается браузерная smoke-проверка на `preview.demoData`. Набор маршрутов берётся из `preview.routes` или строится Core из возможностей манифеста и должен покрывать:

```
стартовая страница
content.intro
content.info
question.single
question.multiple
question.matching
question.ranking
content.summary
results
system.locked, если объявлен
topicNav, если объявлен
progress.active
кнопки навигации/действий
загрузка template.js
```

Критерии успешного прохождения:

```
нет необработанных ошибок
next/answer-submit/test-finish привязаны
ответы сохранены в состояние Core
страница результатов открывается
```

Незаполненный слот критерием прохождения не является: он даёт предупреждение и fallback экрана на стандартный шаблон (§17.1).

Реализация. Фактическая smoke-проверка рендерит каждый маршрут из `preview.routes` на демо-данных общим рендерером и помечает экран проваленным при исключении рендера, пустом результате или `console.error` (`console.warn` — предупреждение). Отсутствующий слот и необъявленный макет дают предупреждение: если хосту передан вход `fallbackLayouts`, экран рендерится из стандартного шаблона. Дополнительно компилируется `template.js`. Привязка `next/answer-submit/test-finish` критерием прохождения не является (относится к перспективным расширенным интерактивам, §3).

Сверх маршрутов `preview.routes` smoke-проверка рендерит ОТЧЁТ — по одному варианту каждого объявленного вида (§8.4). В `preview.routes` отчёт не значит: это не экран прохождения, а документ, — но ломается он так же, а замечает автор только тогда, когда обучающийся не смог скачать PDF. Ошибка отрисовки макета отчёта блокирует активацию наравне с ошибкой экрана.

Реализация. Данные отчёта в проверке строит тот же построитель, что и предпросмотр в настройках теста, на структуре демонстрационного набора (`preview.demoData` : название и темы). Берётся исход «не пройден» — на нём страница показывает больше: вердикт, непройденные темы и блок рекомендаций, то есть отрисовывается больше макета.

Загрузка/обновление шаблона также должны предоставлять:

- статический предпросмотр из `assets.preview` ;
- живой предпросмотр на `preview.demoData` в админском UI.

Авторский UI может показывать статический предпросмотр в галерее и живой предпросмотр в деталях.

18. Обработка runtime-ошибок

SCORM-пакет включает минимальный экран ошибки Core, независимый от выбранного шаблона.

Если рендеринг шаблона или runtime шаблона падает после экспорта, Core показывает аварийный экран:

Не удалось отобразить страницу курса.
Код ошибки: `TEMPLATE_RENDER_ERROR`
Попробуйте обновить страницу или обратитесь к администратору.

Core должен логировать детали в:

- консоль браузера;
- диагностику `suspend_data` , где возможно;
- `telemetry`, если включена.

Пакет не включает стандартный шаблон целиком: в него добавляются только резервные макеты и стили стандартного шаблона для системных экранов и видов отчёта, не объявленных выбранным шаблоном.

19. Зона ответственности SCORM

Поддерживаемые SCORM-операции шаблона идут через `TestBuilder.scorm` .

Core владеет:

- стандартными полями `score`;
- `completion/success status`;

- стандартными интерактивами вопросов;
- публикацией пользовательских показателей результата;
- слиянием `suspend_data` ;
- telemetry payload при завершении.

Шаблонный код (`template.js`) может запрашивать поддерживаемую SCORM-операцию:

```
TestBuilder.scorm.commit();
```